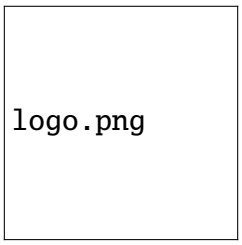
Mar Athanasius College of Engineering
Kothamangalam
(Government Aided Autonomous Institution)



Department of Computer Applications

MINI PROJECT REPORT

ON

AI Forensic Insider Risk Review System

Submitted by

HRISHIKESH P S

Register No.: MCA25MCA-2029

Under the guidance of

Dr. Manu John

2025–2027

Mar Athanasius College of Engineering Kothamangalam

(Government Aided Autonomous Institution)



CERTIFICATE AI Forensic Insider Risk Review System

Certified that this is the bonafide record of the mini project work carried out by

HRISHIKESH P S

Register No.: MCA25MCA-2029

during the academic year 2025–2027, in partial fulfilment of the requirements for the award of the degree of

Master of Computer Applications

under APJ Abdul Kalam Technological University.

Dr. Manu John
Project Guide

Dr. Sonia Abraham
Head of the Department

Project Coordinator
Department of Computer Applications

External Examiner

Acknowledgement

I express my sincere gratitude to the management of Mar Athanasius College of Engineering, Kothamangalam, for providing the academic environment and facilities required to undertake this mini project. The opportunity to study data preparation, machine learning, software design, and responsible review workflows in one project has been an important part of my learning.

I am grateful to my project guide, Dr. Manu John, for guidance, constructive feedback, and encouragement during the planning and development of the AI Forensic Insider Risk Review System. I thank Dr. Sonia Abraham, Head of the Department of Computer Applications, and the project coordinators and faculty members for their support and academic direction.

I also acknowledge the creators and maintainers of the CERT Insider Threat Test Dataset and the developers of the open-source libraries used in this project. The public synthetic dataset made it possible to explore a sensitive research problem without using real employee records. Finally, I thank my family and friends for their encouragement throughout the project.

Abstract

Insider-risk review is difficult because potentially relevant activity is distributed across multiple systems, the number of routine events is large, and known incident examples are rare. This mini project presents Forensio, a local-first review system that aggregates logon, removable device, file, email, and web activity into employee-day features and uses four different model perspectives to prioritize records for human inspection. The implementation combines a Python data and scoring pipeline, a FastAPI service, and a React review dashboard; a Streamlit interface remains available as an alternative.

The model-development study uses the CERT r4.2 synthetic insider-threat dataset. Its saved labelled table contains 330,452 employee-days for 1,000 synthetic users, of which 1,364 employee-days overlap a listed incident interval under the project's labeling rule. Activity events are cleaned and aggregated by user and calendar day. A grouped train/validation/test design keeps users disjoint between partitions. The notebook compares Logistic Regression, Decision Tree, Random Forest, and Isolation Forest, selects variants and a review capacity using validation data, and evaluates the selected operating point on a held-out test partition. In the saved one-percent review-capacity evaluation, the Random Forest has precision 0.144, recall 0.329, F1 0.200, and average precision 0.158. The dashboard separately combines within-dataset percentile ranks and applies an employee-level queue rule.

The system is a research and demonstration prototype, not an investigative finding engine. A high rank is neither a probability nor evidence of misconduct. The dataset is synthetic, labels are incident-window proxies, queue rules are relative to a selected dataset, and the current features do not model employee roles, schedules, or context. Human review, corroborating evidence, access controls, and documented governance are therefore essential before any operational use.

List of tables

1.1 Project components and their responsibilities 3

2.1 Literature themes and their application to this project 5

3.1 Activity-source inventory and the information used 7
3.2 Saved data-cleaning summary 8
3.3 Preprocessing decisions and their consequences 9
3.4 Employee-day feature dictionary 10
3.5 Estimator configuration represented in the notebook 15
3.6 Feasibility summary 20
3.7 Actors and responsibilities 21
3.8 Functional and non-functional requirements 21

4.1 Logical data entities 28
4.2 Local API endpoints 28
4.3 User-interface states and visible response 31

5.1 Saved model comparison at the validation-selected one-percent queue capacity 32
5.2 Random Forest confusion matrix at the one-percent test queue 33
5.3 Random Forest comparison at a five-percent capacity reference 33
5.4 Traceability from project goals to implementation evidence 36

6.1 Demonstration startup sequence 37
6.2 Deployment risks and practical mitigations 39

7.1 Selected repository milestones 40

9.1 Prioritised future work 42

10.1 Software needed for the current demonstration 44
10.2 Practical workstation guidance, not a measured benchmark 45
10.3 Input contract checklist 45
10.4 Functional and data-quality test plan 46
10.5 Saved one-percent test confusion counts by model 47
10.6 Common demonstration issues 48
10.7 Terms used in the report 49

List of figures

3.1 Retained activity events by source in the notebook analysis 11
3.2 Daily recorded activity volume across the dataset period 12
3.3 Ten synthetic users with the largest aggregate event counts 13
3.4 Model-evaluation visualization saved by the analysis notebook 14
3.5 Supervised training and evaluation flow 16
3.6 Dashboard scoring and employee-level prioritisation 17
3.7 End-to-end project pipeline 18

4.1 Application component architecture 24
4.2 Primary analyst use cases 25
4.3 Analysis and review activity 26
4.4 High-level sequence for a folder analysis 27
4.5 Forensio React dashboard landing state 29
4.6 Expanded model guide and scoring explanation 30

Contents

Acknowledgement i

Abstract ii

1 Introduction 1
- 1.1 Background . . . 1
- 1.2 Problem statement . . . 1
- 1.3 Objectives of the project . . . 2
- 1.4 Scope of the project . . . 2
- 1.5 Project domain and technology stack . . . 2
- 1.6 Organisation of the report . . . 3

2 Supporting literature 4
- 2.1 Literature review . . . 4
- 2.2 Literature review summary . . . 5
- 2.3 Findings and proposals . . . 5

3 System analysis 7
- 3.1 Analysis of the dataset . . . 7
  - 3.1.1 About the dataset . . . 7
  - 3.1.2 Explore the dataset . . . 7
  - 3.1.3 Dataset limitations . . . 8
- 3.2 Data preprocessing . . . 8
  - 3.2.1 Data cleaning . . . 8
  - 3.2.2 Analysis of feature variables . . . 9
  - 3.2.3 Analysis of class variables . . . 10
- 3.3 Data visualization . . . 11
- 3.4 Analysis of architecture and algorithms . . . 14
  - 3.4.1 Detailed study of the proposed algorithms . . . 14
  - 3.4.2 Step-by-step explanation of model operation . . . 15
  - 3.4.3 Algorithm flow diagrams . . . 16
- 3.5 Project pipeline . . . 18
  - 3.5.1 Explanation of pipeline stages . . . 18
- 3.6 Feasibility analysis . . . 19
  - 3.6.1 Technical feasibility . . . 19
  - 3.6.2 Economic feasibility . . . 19
  - 3.6.3 Operational feasibility . . . 19
- 3.7 System environment . . . 20

3.7.1 Software environment . . . 20
3.7.2 Hardware environment . . . 20
3.8 Actors and roles . . . 21
3.9 Functional and non-functional requirements . . . 21

4 System design **23**
4.1 Model building . . . 23
4.1.1 Implementation code . . . 23
4.1.2 Model planning and training/testing split . . . 23
4.1.3 Training . . . 23
4.1.4 Testing . . . 24
4.2 System architecture . . . 24
4.2.1 Component responsibilities . . . 24
4.3 Use-case model . . . 25
4.4 Activity diagram . . . 26
4.5 Sequence diagram . . . 27
4.6 Data design . . . 27
4.7 API design . . . 28
4.8 UI design . . . 29
4.8.1 Interaction states . . . 31
4.8.2 Accessibility and usability . . . 31

5 Results and discussion **32**
5.1 Testing approach . . . 32
5.2 Model planning and evaluation protocol . . . 32
5.3 Comparison of models . . . 32
5.4 Evaluation metrics . . . 34
5.5 Dashboard consensus and offline evaluation . . . 34
5.6 Confusion matrix interpretation . . . 34
5.7 Discussion . . . 35
5.8 Requirements traceability . . . 36

6 Model deployment **37**
6.1 Local demonstration setup . . . 37
6.2 Model implementation . . . 37
6.3 Analyst workflow . . . 38
6.4 Operational limitations . . . 38
6.5 Deployment risk review . . . 39

7 Git history **40**

8 Conclusions **41**

8.1 Summary of the project 41
8.2 Learning outcomes 41
8.3 Final conclusion 41

9 Future scope **42**
9.1 Model and evaluation improvements 43
9.2 Software and governance improvements 43

10 Appendix **44**
10.1 Minimum software requirements 44
10.2 Minimum hardware requirements 44
10.3 Input contract and data lineage 45
10.4 Test cases and verification plan 46
10.5 Evaluation metrics and saved counts 47
10.6 Troubleshooting guide 48
10.7 Responsible-use checklist 48
10.8 Glossary 49
10.9 Reproducibility checklist 49
10.10 Notebook source listing 50
10.11 Scoring core source listing 66
10.12 FastAPI source listing 71
10.13 Streamlit source listing 75
10.14 React application source listing 79
10.15 Interface style source listing 85

11 References **99**

1. Introduction

1.1 Background

Modern organisations generate activity records through identity providers, endpoints, file services, email systems, and web gateways. These records are useful for security operations, but their volume and fragmentation make manual review difficult. A security analyst may need to correlate a sequence of events across several sources before deciding whether it warrants escalation. The challenge is not simply to detect a single unusual event; it is to organise context in a way that supports a careful and reproducible investigation.

Insider-risk analysis concerns activity associated with trusted accounts and authorised access. A meaningful review must distinguish between a statistical signal and a conclusion about a person. Legitimate work can produce unusual activity, while harmful activity may resemble normal operations. Differences in job role, work schedule, travel, project deadlines, and system configuration can all affect the observed event pattern. For that reason, a decision-support tool should make its inputs and ranking logic understandable and should preserve human authority over all consequential decisions.

This project implements a prototype called Forensio. It transforms five activity logs into employee-day counts, scores those feature rows using four model families, and displays a ranked review queue. It is designed to demonstrate a technical workflow on the CERT r4.2 synthetic dataset. It is not a production surveillance platform, does not inspect message or file content, and does not establish intent.

1.2 Problem statement

The project addresses the following bounded problem: given a local folder containing the five supported activity CSV files and a previously trained model bundle, produce an employee-day and employee-level ranking that helps an analyst decide which records to review first. The system must integrate disparate sources, handle large CSV files with bounded-memory processing, apply the same feature contract used in training, expose clear model context, and avoid presenting a score as proof.

The study also evaluates whether a set of supervised and unsupervised models can rank synthetic incident-window employee-days above other days when users are kept separate across development and test partitions. A low base rate makes accuracy alone unsuitable as the primary measure. The study therefore reports average precision and capacity-based precision, recall, and F1 alongside the confusion counts.

1.3 Objectives of the project

The objectives are to:

1. examine the supported activity data and its quality;
2. build a repeatable employee-day aggregation and labeling workflow;
3. compare four distinct model families under user-grouped evaluation;
4. build a local scoring service that reuses the trained feature schema;
5. create a dashboard for selecting data, following analysis progress, and reviewing ranked employees and activity details;
6. document evaluation limits, privacy considerations, and responsible-use controls.

1.4 Scope of the project

The project scope includes synthetic logon, device, file, email, and HTTP event records; aggregation at one calendar day per user; the feature columns persisted with the trained bundle; offline model evaluation; local scoring; and a human-readable review interface. Optional employee-name lookup files are supported by the local scoring code, but names are not required for model input.

The current scope excludes live collection agents, enterprise identity integrations, cross-organisation model transfer, streaming analysis, automated disciplinary decisions, legal attribution, message-body or file-content inspection, calibrated probability estimates, and production authentication or role-based access control. A dashboard rank is relative to the data batch being scored. The offline employee-day test results do not validate the dashboard's employee-level status rule.

1.5 Project domain and technology stack

The project combines data engineering, applied machine learning, information visualization, and digital-forensics workflow concepts. Python and pandas perform input handling and aggregation; scikit-learn supplies estimators and metrics; joblib persists the model bundle; FastAPI exposes a local HTTP interface; and React, JavaScript, Vite, and CSS provide the browser dashboard. Streamlit is retained as an alternative interface. CSV files are used for both inputs and result exports; the prototype does not require a database.

Table 1.1: Project components and their responsibilities

Component	**Technology**	**Responsibility**
Data preparation	Python, pandas, NumPy	Clean activity records, aggregate features, and align model columns.
Model development	scikit-learn	Fit and evaluate supervised and unsupervised estimators.
Model persistence	joblib	Store estimators and the feature-column contract for local reuse.
Service layer	FastAPI, Pydantic	Validate requests, manage analysis endpoints, and return JSON results.
Web interface	React, Vite, CSS	Select log folders, display progress, search and filter the review queue.
Alternative interface	Streamlit	Provide a Python-only local demonstration path.
Input and outputs	CSV	Interchange activity logs, scores, and report tables.

1.6 Organisation of the report

Chapter 2 reviews insider-risk analytics and relevant evaluation principles. Chapter 3 describes the dataset, preprocessing, visualization, algorithms, pipeline, feasibility, and operating environment. Chapter 4 presents model construction, software architecture, requirements, data design, and interface design. Chapter 5 reports the saved model evaluation and discusses its limits. Chapter 6 describes the local deployment and analyst workflow. Chapter 7 summarises the repository history. Chapters 8 and 9 give the conclusions and future work. Chapter 10 contains reproducibility information, requirements, test cases, data definitions, responsible use guidance, and source listings. Chapter 11 provides references.

2. Supporting literature

2.1 Literature review

Insider-threat research has long treated the problem as a combination of behavioural, organisational, and technical factors rather than a single signature. The CERT Insider Threat Test Dataset provides synthetic scenarios and activity records intended to support repeatable research on this subject [1]. Synthetic examples are valuable for development because they permit experimentation without exposing real employee data. They also create an important limitation: generated behaviour is not a substitute for evidence collected in a real organisation, and performance on one release cannot be assumed to transfer to another population.

The project follows a common feature-engineering pattern: transform event-level data into a table whose rows represent a user and a calendar day. Aggregation reduces the number of records and makes multi-source activity easier to compare. Count and timing features can reveal volume changes or activity outside a fixed interval, but they lose event ordering and detailed context. A day-level table cannot reconstruct a forensic timeline. The system therefore treats the aggregate as a navigation aid and directs an analyst back to original records for validation.

The algorithm set deliberately includes contrasting assumptions. Logistic Regression is a linear supervised classifier and provides a relatively simple baseline. A Decision Tree expresses threshold relationships through a sequence of splits but can overfit if unconstrained. A Random Forest averages many trees to reduce the instability of a single tree. Isolation Forest estimates unusualness without using incident labels [4]. These algorithms do not produce interchangeable quantities: a class probability, a decision function, and an anomaly score have different semantics. This project converts each output to a percentile rank within the current scoring batch before combining dashboard scores.

Rare labels and high class imbalance are central evaluation concerns. When a positive example is a small fraction of a test set, a model can achieve high accuracy by ranking nearly all records as negative. Precision-recall measures are more informative for a review queue because they describe the fraction of selected records matching known positive labels and the fraction of known positives retrieved. Average Precision summarises precision over score thresholds. A fixed-capacity queue also corresponds to an operational constraint: an analyst can review only a limited number of records in a given period. Evaluation must specify whether capacity selection was performed on validation data or the test set.

User-level leakage is another important issue in behavioural data. Repeated daily records from one person share stable characteristics. If records are randomly distributed across training and testing, the model may benefit from seeing a user's other days before evaluation. The notebook therefore uses stratified group splitting with the user identifier as the group. The

identifier is not a model feature; it is used to keep users in one partition.

Responsible monitoring practices require a clear purpose, access limits, proportionality, retention controls, and a review process. Security frameworks such as NIST SP 800-53 describe system and information security controls, while NIST SP 800-61 describes incident response practices [2, 3]. These documents are not specific model-governance standards, but they provide useful control contexts for access, logging, incident handling, and evidence preservation. A risk score alone should not trigger an adverse action.

2.2 Literature review summary

Table 2.1: Literature themes and their application to this project

Theme	**Practical implication**	**Project response**
Synthetic research data	Enables reproducible experimentation but may not represent real workplaces.	Use CERT r4.2 and explicitly limit conclusions to the saved synthetic evaluation.
Event aggregation	Simplifies modelling but removes sequence and context.	Aggregate to employee-day counts and preserve source records for follow-up.
Rare positive labels	Accuracy can conceal low precision and missed cases.	Report AP and capacity-based precision, recall, F1, and confusion counts.
Repeated observations	Random row splits can leak individual patterns.	Keep employees disjoint across training, validation, and test partitions.
Human investigation	Model output is a lead rather than a finding.	Display review language and require independent analyst verification.

2.3 Findings and proposals

The reviewed concepts motivate four design decisions. First, the project uses a multi-source daily representation rather than treating each event as an independent case. Second, it compares model families instead of presuming that one algorithm is universally suitable. Third, it separates model selection from final test reporting. Fourth, it describes the interface as a review aid and makes the relative nature of the scores explicit.

The literature also identifies gaps that remain open. A future evaluation should include time-based testing, additional synthetic releases, calibrated and subgroup-aware analyses where ethically and lawfully appropriate, and structured analyst feedback. Before any real-world use, an organisation would need to define a lawful purpose, consult relevant stakeholders, assess

privacy and employment impacts, set retention limits, control access, and validate the complete workflow rather than only the estimator.

3. System analysis

3.1 Analysis of the dataset

3.1.1 About the dataset

The development notebook reads the CERT r4.2 release from the repository's local `r4.2` folder and the matching `answers/insiders.csv` answer file. The activity sources are `logon.csv`, `device.csv`, `file.csv`, `email.csv`, and `http.csv`. The answer file is used during offline training and evaluation; a dashboard user supplies activity logs only. It is not a live incident feed.

Table 3.1: Activity-source inventory and the information used

Source	Role in the pipeline	Aggregated information
`logon.csv`	Authentication activity	Event count and count before 06:00 or at/after 18:00.
`device.csv`	Removable-device activity	Event count, after-hours count, and connect-event count.
`file.csv`	File-system activity	Event count and after-hours count; file contents are not read.
`email.csv`	Email activity	Event count, after-hours count, and numeric attachment total.
`http.csv`	Web activity	Event count and after-hours count; URL content is not modelled.
`answers/insiders.csv`	Research labels	User, incident interval, scenario, and dataset version for offline labels.

The saved labelled employee-day table contains 330,452 rows for 1,000 synthetic users. Its calendar coverage runs from 2 January 2010 through 17 May 2011. The project labeling rule marks 1,364 employee-days as positive, or approximately 0.413 percent. The positive days belong to 70 user identifiers. These values describe the saved project artifacts; they are not estimates of prevalence in a real workforce.

3.1.2 Explore the dataset

The five source files contain 32,770,222 raw event rows in total. The recorded cleaning summary retains 32,409,354 rows and removes 360,868 duplicate rows. No missing user identifiers or invalid dates are reported in those saved summaries. Most records are in the HTTP source, so a single source can dominate an event-volume plot. The features used by the model are counts,

not concatenated raw event histories.

Table 3.2: Saved data-cleaning summary

Source	**Rows read**	**Rows kept**	**Duplicates removed**	**Rows dropped**
Logon	854,859	854,859	0	0
Device	405,380	405,374	6	6
File	445,581	445,581	0	0
Email	2,629,979	2,618,056	11,923	11,923
HTTP	28,434,423	28,085,484	348,939	348,939
Total	**32,770,222**	**32,409,354**	**360,868**	**360,868**

The cleaning report counts duplicate removals within the implemented chunk-processing workflow. Deduplication performed independently inside separate chunks may not remove a duplicate whose copies fall in different chunks. Therefore, the saved counts should be interpreted according to the implementation rather than as proof of global, cross-chunk uniqueness. If global deduplication is required, the pipeline should be extended and its memory and identity semantics specified.

3.1.3 Dataset limitations

CERT r4.2 is a synthetic benchmark. Its events, scenarios, identities, and incident windows are useful for repeatable experiments but do not represent a verified real employee population. There is no guarantee that the synthetic incident patterns cover all plausible behaviours, that background distributions match another organisation, or that a trained score will generalise across job roles and time periods.

The five activity tables are not necessarily complete observations of all employee behaviour. Missing sources, source-specific collection gaps, clock differences, account-sharing, and system changes can all alter counts. The prototype currently assumes that timestamps can be parsed and converted into a calendar day without a configured organisation-specific time zone. These assumptions should be checked before a different dataset is analysed.

3.2 Data preprocessing

3.2.1 Data cleaning

The production scoring core reads each CSV in chunks of 25,000 rows. Column names are normalised to lower case and stripped of surrounding spaces. The parser requires usable `date` and `user` columns, trims user identifiers, parses dates with invalid values coerced to missing,

removes rows with unusable users or dates, and deduplicates rows within a chunk. It then derives a calendar-day key, an after-hours indicator, and any source-specific features. Each source is grouped on user and day before source tables are combined.

After-hours activity is defined mechanically as an event before 06:00 or at/after 18:00. The boundary is inclusive at 18:00. This fixed interval is not adjusted for shifts, daylight-saving interpretation, holidays, travel, or personal schedules. It is a research feature, not a policy determination.

Table 3.3: Preprocessing decisions and their consequences

Decision	**Implementation behaviour**	**Consequence to consider**
Chunked ingestion	Reads up to 25,000 records per chunk.	Bounds the size of an individual read but does not make all later tables constant-memory.
Header normalisation	Trims and lowercases selected field names.	Helps tolerate case and whitespace differences; schema aliases remain limited.
Timestamp parsing	Invalid dates are coerced and excluded from aggregation.	Bad timestamp rows do not contribute; data-quality counts should be reviewed.
Identifier cleaning	User values are converted to strings and trimmed.	Whitespace variants are merged; broader identity resolution is not attempted.
Missing source/day values	Missing feature columns are created or filled with zero.	Zero means no usable count was observed, not confirmed absence of activity.
Duplicate handling	Duplicates within a chunk are removed.	Cross-chunk duplicate removal is not guaranteed by this step.

3.2.2 Analysis of feature variables

The model input is a numeric vector for each employee-day. User ID, date, scenario, names, and labels are not model features. The same feature-column list is stored with the model bundle and re-applied by the scoring service. When a feature expected by the bundle is not present in a newly scored table, the scoring code inserts a zero column. This permits a consistent shape, but it can conceal a missing source unless the operator also reviews input completeness.

Table 3.4: Employee-day feature dictionary

Feature name	Source	Definition
logon_event_count	Logon	Number of retained logon events on the day.
logon_after_hours_count	Logon	Logon events outside the fixed 06:00–18:00 interval.
device_event_count	Device	Number of retained device events.
device_after_hours_count	Device	Device events outside the fixed interval.
device_device_connect_count	Device	Device events whose activity value is connect.
file_event_count	File	Number of retained file events.
file_after_hours_count	File	File events outside the fixed interval.
email_event_count	Email	Number of retained email events.
email_after_hours_count	Email	Email events outside the fixed interval.
email_attachment_count	Email	Sum of numeric attachment values; unparseable values contribute zero.
http_event_count	HTTP	Number of retained web events.
http_after_hours_count	HTTP	Web events outside the fixed interval.

The selected features deliberately omit message bodies, filenames, document contents, and URL strings. This reduces exposure of potentially sensitive content and makes the feature table compact. It also limits explanatory detail: a count cannot say which file was accessed or why a person used a system. The activity detail view exposes selected aggregated counts for review; original records must be examined under appropriate authority when a lead warrants further investigation.

3.2.3 Analysis of class variables

The notebook joins the CERT answer file to employee activity using user identity and incident dates. For each listed incident, the project normalises start and end timestamps to calendar days and labels matching user-days from the start day through the end day, inclusive. A positive label therefore means only that an employee-day overlaps a listed synthetic incident interval. A negative label means that the row was not marked positive under this rule; it is not proof of benign activity.

Day-level labels have known limitations. If an incident interval overlaps one day, all retained events for that user on that day are aggregated together, even if only a subset is related to the scenario. Relevant behaviour outside a listed interval is labelled negative. Thus, the training target is a useful benchmark proxy, not a ground-truth statement about every event or person.

3.3 Data visualization

The notebook contains plots of event counts by source, daily event volume, high-volume synthetic users, and model evaluation outputs. These plots are descriptive summaries. Event volume alone does not represent risk, and users with the largest number of events may simply have different roles or activity levels.

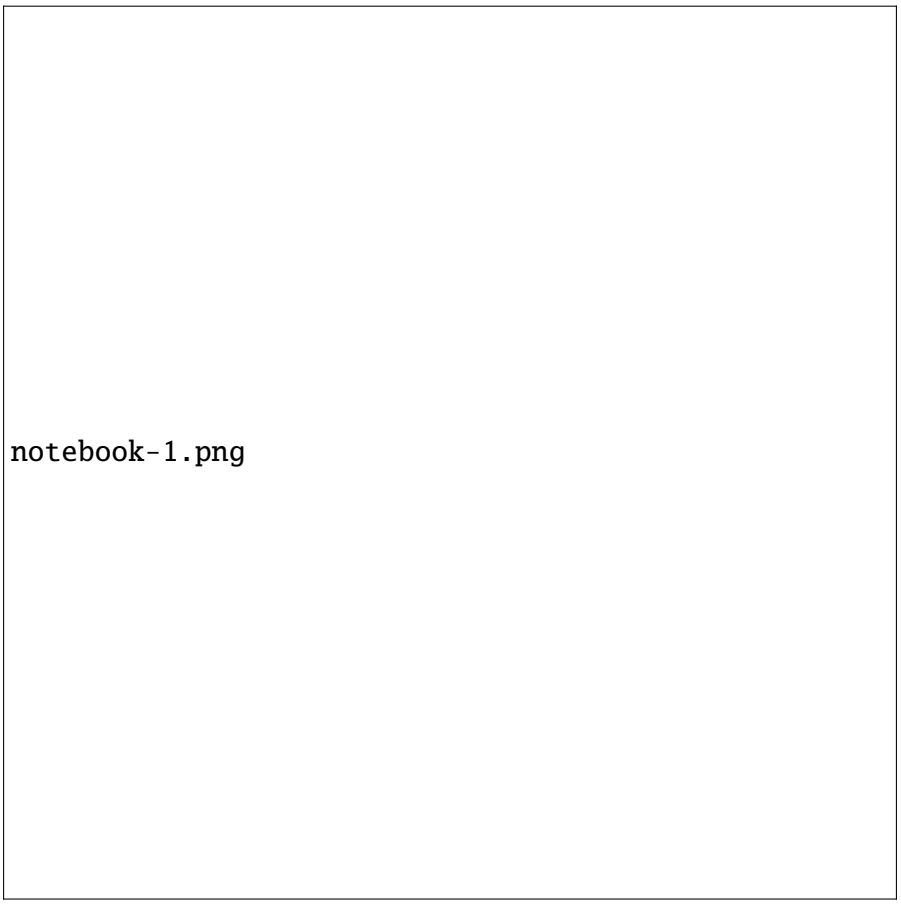


Figure 3.1: Retained activity events by source in the notebook analysis

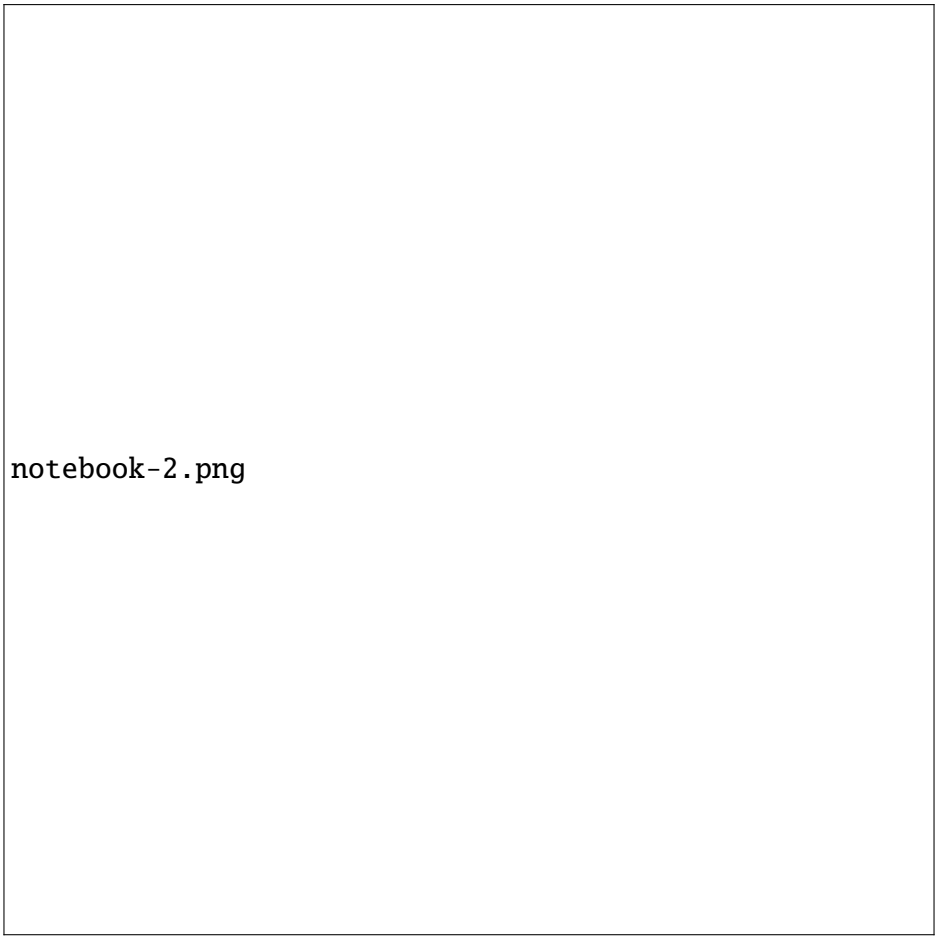


Figure 3.2: Daily recorded activity volume across the dataset period

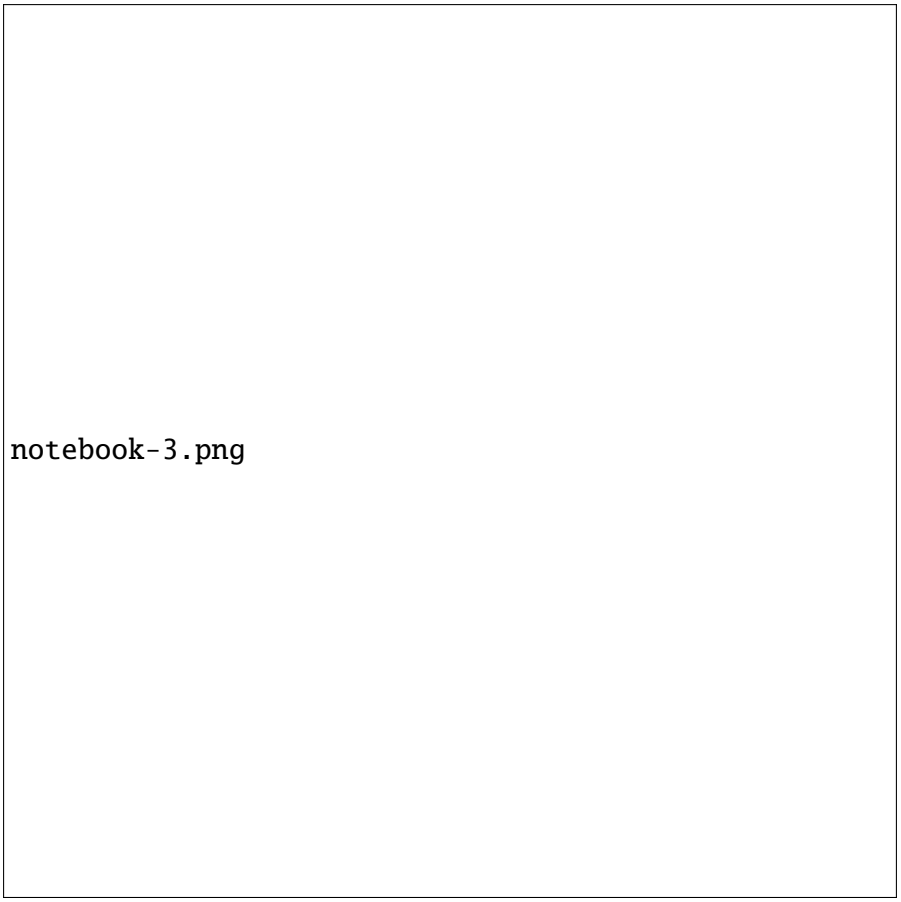


Figure 3.3: Ten synthetic users with the largest aggregate event counts

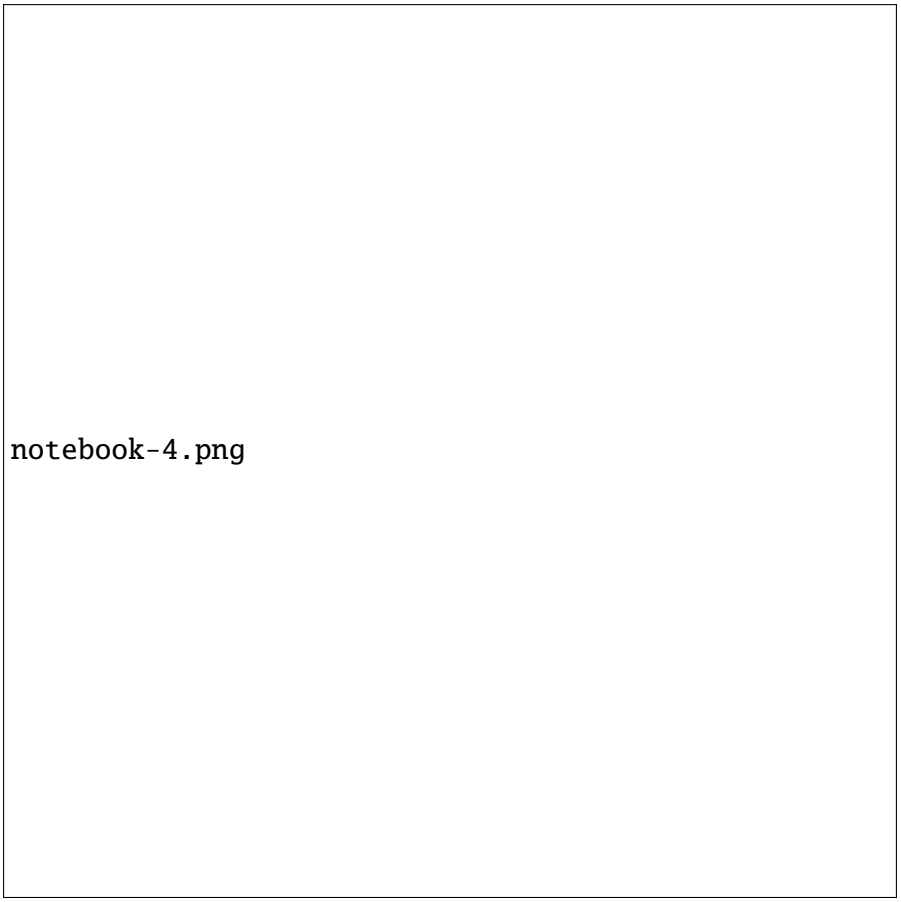


Figure 3.4: Model-evaluation visualization saved by the analysis notebook

The event-source plot shows why counts should be compared within an appropriate feature and context. The daily series illustrates that aggregate volume changes across the dataset period. The top-user chart is not a list of suspicious employees. The evaluation visualization must be read with the positive-class support and the review-capacity rule; an attractive aggregate score cannot remove label uncertainty.

3.4 Analysis of architecture and algorithms

3.4.1 Detailed study of the proposed algorithms

Logistic Regression. The pipeline applies median imputation, standard scaling, and a class-weighted Logistic Regression estimator. The maximum iteration count is 1,000 and the random seed is fixed. Scaling is important for a linear estimator when input counts have different magnitudes. Class weighting gives the rare positive class greater influence during fitting, but it does not by itself guarantee high precision.

Decision Tree. The tree uses a maximum depth of 10, a minimum leaf size of 10, balanced class weights, and a fixed random seed. Each path through the tree applies feature thresholds. This can be easier to explain than a large ensemble, but the model can still learn patterns specific

to its training data and does not provide causal explanations.

Random Forest. The forest contains 200 decision trees, uses a minimum leaf size of 5, and applies `balanced_subsample` class weighting. Bootstrap variation and aggregation can reduce the variance of an individual tree. In the notebook, a real-data baseline is compared with an augmented candidate. The augmented candidate is selected only when it improves validation Average Precision. The saved comparison indicates 815 synthetic rows were added to the selected Random Forest.

Isolation Forest. This model uses 200 estimators and automatic contamination handling. It is fitted without incident labels and assigns a lower decision function to more anomalous observations; the pipeline reverses the sign so that larger values indicate greater unusualness. Anomaly is not synonymous with harmful conduct, and this model's score is not a probability.

Table 3.5: Estimator configuration represented in the notebook

Model	**Configuration**	**Output used**
Logistic Regression	Median imputation; standard scaling; class-weighted; 1,000 maximum iterations; seed 42.	Positive-class prediction score.
Decision Tree	Maximum depth 10; minimum leaf size 10; balanced weights; seed 42.	Positive-class prediction score.
Random Forest	200 trees; minimum leaf size 5; balanced-subsample weights; parallel fitting; seed 42.	Positive-class prediction score; validation-selected baseline or augmented variant.
Isolation Forest	200 trees; automatic contamination; parallel fitting; seed 42.	Negative decision-function value, oriented so larger is more unusual.

3.4.2 Step-by-step explanation of model operation

The data flow begins with a feature matrix X, binary incident-window labels y, and a user-group vector g. A stratified group split creates training, validation, and test partitions without sharing user identifiers between them. Supervised models fit on training features and labels. Isolation Forest fits only on training features.

For each candidate, a risk score is computed for validation data. Random Forest compares a real-data fit to an augmented fit, where positive training rows are resampled and count-perturbed. Validation Average Precision selects between these variants. The notebook then chooses a review capacity by searching candidate rates using validation F1. Only after selection are test metrics calculated. The test set is not used to choose the estimator or the queue rate.

1. Construct and validate the employee-day feature matrix.
2. Split groups of employees into training, validation, and test partitions using the notebook's

stratified group splitters.

3. Fit the four model families on training data, with labels supplied only to supervised models.
4. Compare the Random Forest's baseline and augmented variants using validation Average Precision.
5. Convert each score into a fixed-capacity decision for validation experiments and select the queue rate by validation F1.
6. Evaluate selected estimators and operating capacity on untouched test data; retain both ranking metrics and confusion counts.
7. Persist estimator objects, selected feature columns, and metadata in the model bundle for dashboard scoring.

3.4.3 Algorithm flow diagrams

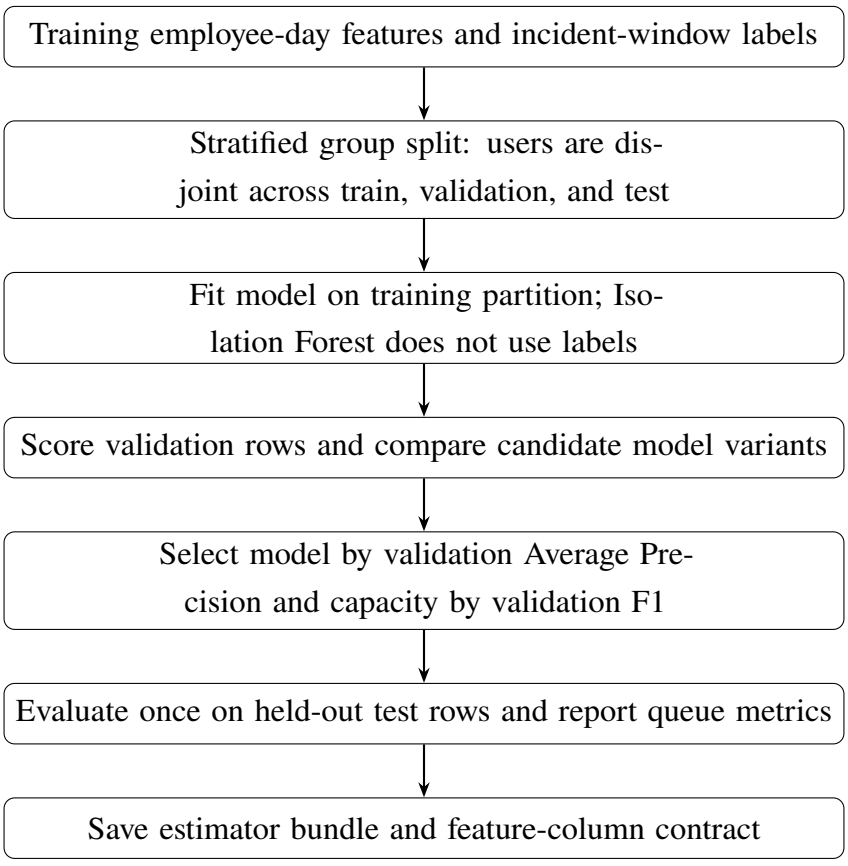


Figure 3.5: Supervised training and evaluation flow

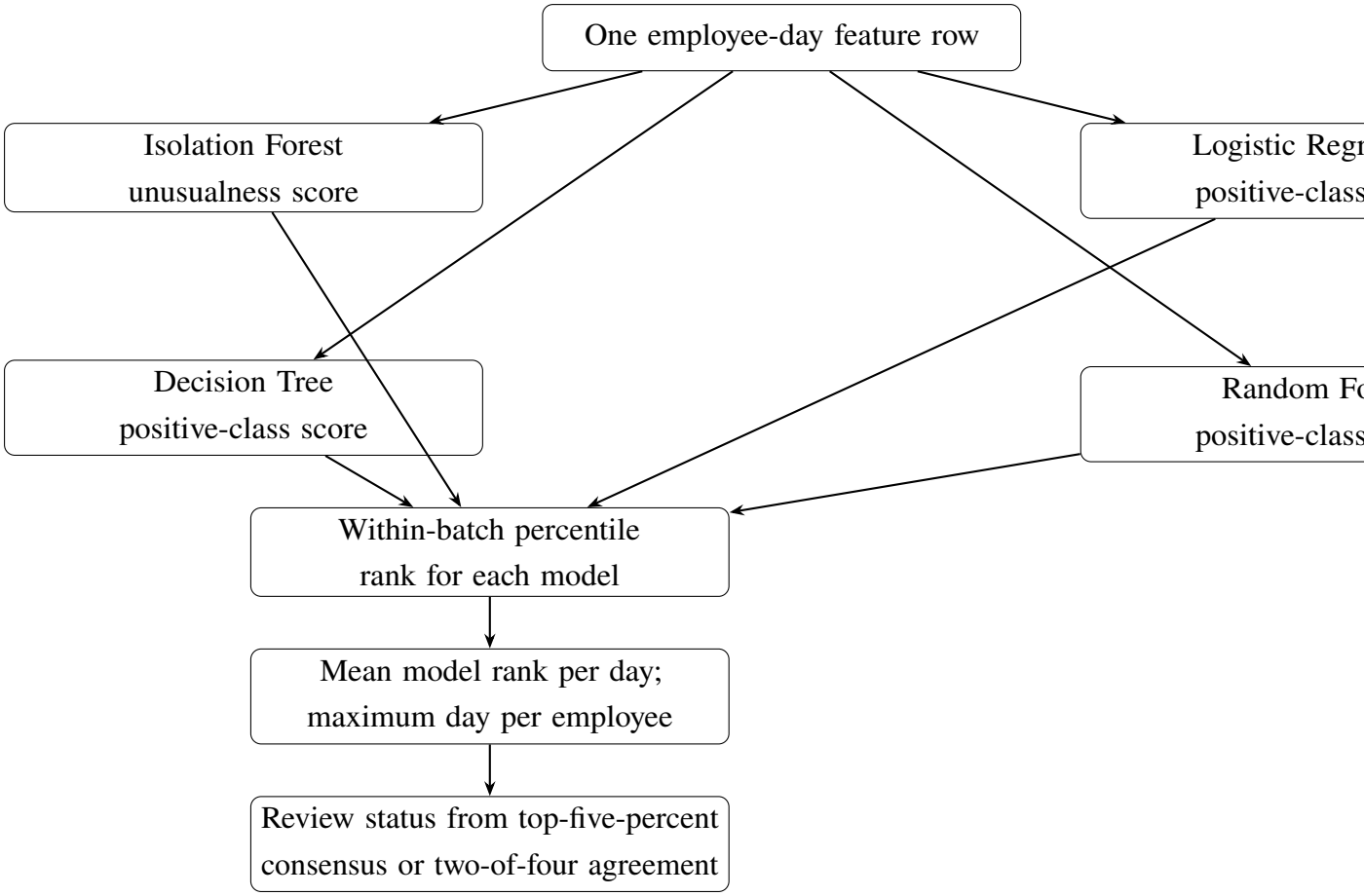


Figure 3.6: Dashboard scoring and employee-level prioritisation

The dashboard first creates percentile ranks separately within the current scoring batch because the model outputs are on different scales. A per-day consensus is the mean of the four ranks. The employee table uses the highest daily consensus for each user and each model's highest employee-level rank. The display status is set when consensus is within the top five percent of employees or at least two model ranks are in their respective top ten percent. This rule is intended to prioritise review, not to classify guilt.

3.5 Project pipeline

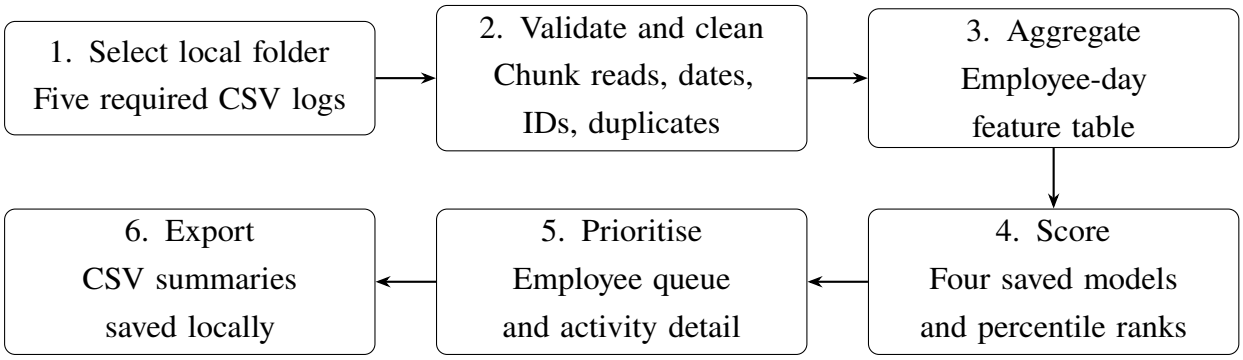


Figure 3.7: End-to-end project pipeline

3.5.1 Explanation of pipeline stages

Input selection. The React client gathers files from a selected folder and posts the expected CSVs to the local API. An alternative API endpoint accepts a folder path on the host. The application expects the five source filenames exactly, apart from case normalisation in upload handling.

Validation and cleaning. Required source presence is checked before scoring. Each log is streamed in chunks and normalised to the columns the aggregator needs. Missing required columns or unreadable records cause explicit errors rather than a successful-looking empty report.

Aggregation and alignment. Each source becomes a user-day table. Tables are aligned on user and day, missing source-day contributions are filled with zero, and the final feature matrix is reordered to the saved feature list. This ordering is critical because estimators bind learned parameters to column positions.

Scoring and ranking. Every estimator processes the same feature matrix. Scores are converted to percentile ranks, averaged per employee-day, and reduced to employee-level maxima. The system builds human-readable reasons from selected high-volume signals on the highest-consensus day.

Review and export. The dashboard provides a ranked queue, summary metrics, filtering, search, per-model scores, and a short activity table. The API and Streamlit app write day-level and employee-level CSV outputs into `sprint1_output`. The outputs are local demonstration artifacts and are not an immutable evidence store.

3.6 Feasibility analysis

3.6.1 Technical feasibility

The system is technically feasible as a local prototype because its components use widely available Python and JavaScript tooling, its inputs are ordinary CSV files, and model training can be performed offline. The scorer limits individual CSV reads to 25,000-row chunks. The overall workflow still holds grouped and scored data in memory, so the project has not demonstrated arbitrary-scale streaming. Runtime and memory should be benchmarked on the target machine and data volume.

3.6.2 Economic feasibility

The software stack relies on open-source packages and a local development environment. No hosted service, paid API, or dedicated database is required for the demonstration. Costs remain in personnel time, workstation resources, data governance, security review, maintenance, and any organisational licensing or infrastructure requirements. Open-source availability does not eliminate the cost of operating a responsible investigation process.

3.6.3 Operational feasibility

An analyst can select a folder, run analysis, inspect a queue, and review activity aggregates without learning the model code. The prototype provides an understandable entry point, but it does not support case assignment, audit trails, reviewer notes, approvals, or incident-system integration. Operational deployment would require those controls and a written process for challenging, documenting, and closing a review.

Table 3.6: Feasibility summary

Dimension	Current position	Condition for broader use
Technical	Local Python/API/browser stack and serialized models work as a coherent prototype.	Benchmark full data volume; add schema validation, failure recovery, and monitoring.
Economic	No external paid service is required for the demonstration.	Budget for review staff, secure endpoints, maintenance, and governance.
Operational	Folder selection and ranked review are straightforward.	Add case workflow, auditability, accountable reviewers, and escalation policy.
Privacy	Feature generation omits content fields and processing is local by design.	Enforce access, retention, consent/legal basis, and secure handling of source logs.

3.7 System environment

3.7.1 Software environment

The Python backend depends on pandas, NumPy, scikit-learn, joblib, FastAPI, Pydantic through FastAPI, python-multipart, and Uvicorn. Streamlit is used by the alternative interface. The browser client uses React and Vite. Exact dependency versions should be captured in a lock file for a reproducible deployment; the repository's current requirements file lists package names without strict versions.

3.7.2 Hardware environment

The demonstrated setup is a developer workstation capable of running Python, a local web service, and a browser. The saved model bundle and large data files determine the actual memory and storage requirements. No measured minimum CPU, RAM, disk, or runtime benchmark is included in the repository; the hardware values in Appendix 10.1 are therefore presented as practical setup guidance, not measured guarantees.

3.8 Actors and roles

Table 3.7: Actors and responsibilities

Actor	Responsibilities	System boundary
Analyst / operator	Select a dataset, run analysis, inspect ranked signals, verify source evidence, document a review.	The prototype supports selection and inspection; investigation decisions remain external.
Data custodian	Authorise use of input logs and ensure correct scope and retention.	Not implemented as an application role; required operational control.
Model maintainer	Build and version the model bundle, record evaluation and feature schema, assess changes.	Notebook and model artifacts support development; no registry is present.
Application operator	Start the local API and browser client, monitor errors, protect outputs.	Local developer workflow; production service controls are out of scope.

3.9 Functional and non-functional requirements

Table 3.8: Functional and non-functional requirements

ID	Requirement	Class	Evidence in project
FR-01	Accept the five required activity sources and report missing files.	Functional	API upload validation and scorer file checks.
FR-02	Process records with bounded chunk reads.	Functional	Core CSV processor uses 25,000-row chunks.
FR-03	Reproduce the trained employee-day feature schema.	Functional	Model bundle stores `feature_columns`.
FR-04	Score each employee-day with four estimator families.	Functional	Core scoring loop iterates through model names.
FR-05	Present employee-level priorities and a reason summary.	Functional	API response and dashboard queue.

Continued on next page

ID	Requirement	Class	Evidence in project
FR-06	Show activity detail for a selected employee.	Functional	Employee endpoint returns up to 20 ranked days.
FR-07	Export day-level and employee-level CSV summaries.	Functional	Output files are written to `sprint1_output`.
NFR-01	Keep the demonstration local by default.	Non-functional	API binds to loopback in the documented run configuration.
NFR-02	Distinguish relative ranks from probabilities or proof.	Non-functional	Dashboard text and report disclaimers.
NFR-03	Fail visibly when data or model prerequisites are missing.	Non-functional	API returns errors and UI displays failures.
NFR-04	Preserve understandable source-to-feature lineage.	Non-functional	Source-specific aggregate definitions are documented.
NFR-05	Support reproducible evaluation.	Non-functional	Fixed random seeds and saved evaluation artifacts.
NFR-06	Protect sensitive data at rest and in transit.	Non-functional	Not fully implemented; a deployment prerequisite.

4. System design

4.1 Model building

Model development and model deployment are separate stages. The notebook uses labelled synthetic activity for training and evaluation. The deployed local scorer accepts unlabeled activity, loads an already-created bundle, and produces a ranked report. The dashboard does not retrain estimators when an analyst submits a folder.

4.1.1 Implementation code

Notebook cells implement data inspection, aggregation, label construction, group splitting, model fitting, validation selection, test evaluation, visualisation, and bundle persistence. The scoring core shares the feature-generation contract used by the application. The selected feature list is included in the bundle so the inference path can insert missing columns and order inputs consistently.

4.1.2 Model planning and training/testing split

The notebook applies an outer `StratifiedGroupKFold` splitter with five folds and a fixed seed to hold out test users. An inner four-fold stratified group split with a separate seed partitions the remaining users into training and validation groups. The first split produced by each configured splitter is used. The identifier groups are checked for overlap. If any partition lacks a positive employee-day, the notebook stops rather than reporting an unusable comparison.

The saved evaluation partition contains 66,118 employee-days and 289 positive rows. The group split is a stronger safeguard against same-user leakage than a random row split, though it does not solve temporal drift, release-to-release shift, or differences between synthetic and real activity.

4.1.3 Training

The supervised estimators are fitted on training labels only. Isolation Forest is fitted on training features and does not use the incident labels. The Random Forest augmentation routine creates one count-perturbed positive example per positive training row. It samples positive anchors, applies a log-normal multiplier, resamples event counts from Poisson distributions, and reconstructs after-hours, device-connection, and attachment counts while preserving source-level relationships. Validation Average Precision decides whether the augmented version is retained. Test data is not used for that decision.

4.1.4 Testing

The notebook evaluates the selected estimators on the held-out test partition. It chooses the queue rate on validation data by searching a set of candidate capacities and maximising validation F1. The saved rate is one percent. At that rate the test set contributes 662 employee-days to the queue. A five-percent capacity is also retained as a comparison reference. These are ranking capacities, not fixed probability thresholds.

4.2 System architecture

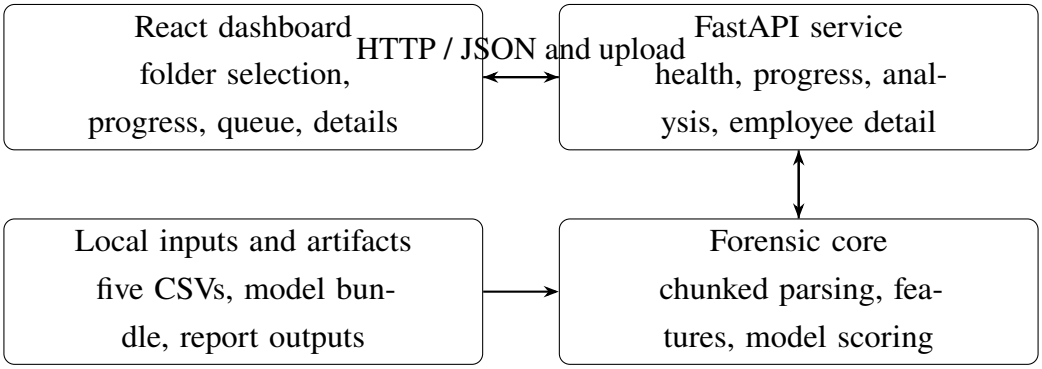


Figure 4.1: Application component architecture

4.2.1 Component responsibilities

React client. The client checks health, collects files using a directory input, uploads the selected CSVs, polls progress, and renders the report. It supports employee search and status filtering, and requests activity detail for a selected employee.

FastAPI service. The service validates uploaded filenames, stages accepted files in a temporary directory, invokes the scoring core, stores the most recent in-memory results for detail requests, and writes CSV exports. It exposes health and progress endpoints for the UI. A lock serialises analysis requests in this single-process prototype.

Forensic core. The core checks model availability and required input files, parses the logs in chunks, builds user-day features, orders model inputs, scores with each estimator, calculates consensus ranks, generates reasons, and formats selected activity rows.

Model bundle. The bundle contains the four fitted models and the feature-column list. It is loaded with joblib. Bundle files are trusted local artifacts; loading joblib from an untrusted source is not safe and must not be permitted in an operational deployment.

4.3 Use-case model

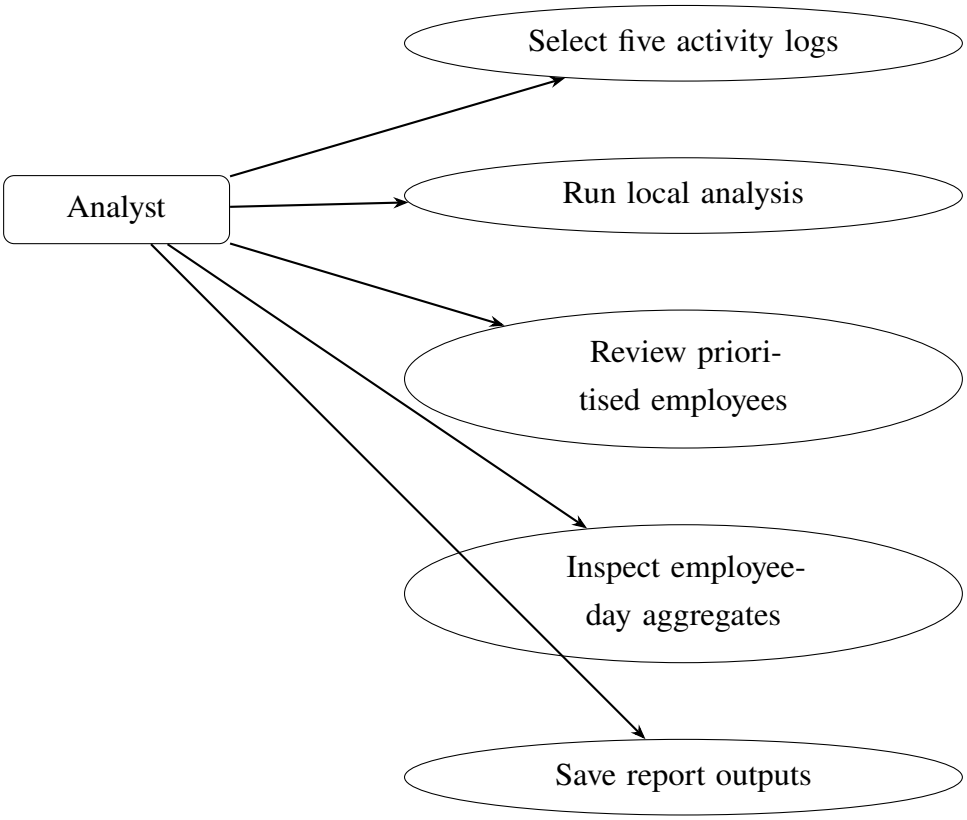


Figure 4.2: Primary analyst use cases

The analyst is the principal interactive actor. Data custodians and model maintainers are operational stakeholders but are not authenticated application roles. The use-case diagram intentionally does not model automated decision-making: the application ranks records and exposes information for a human process outside the prototype.

4.4 Activity diagram

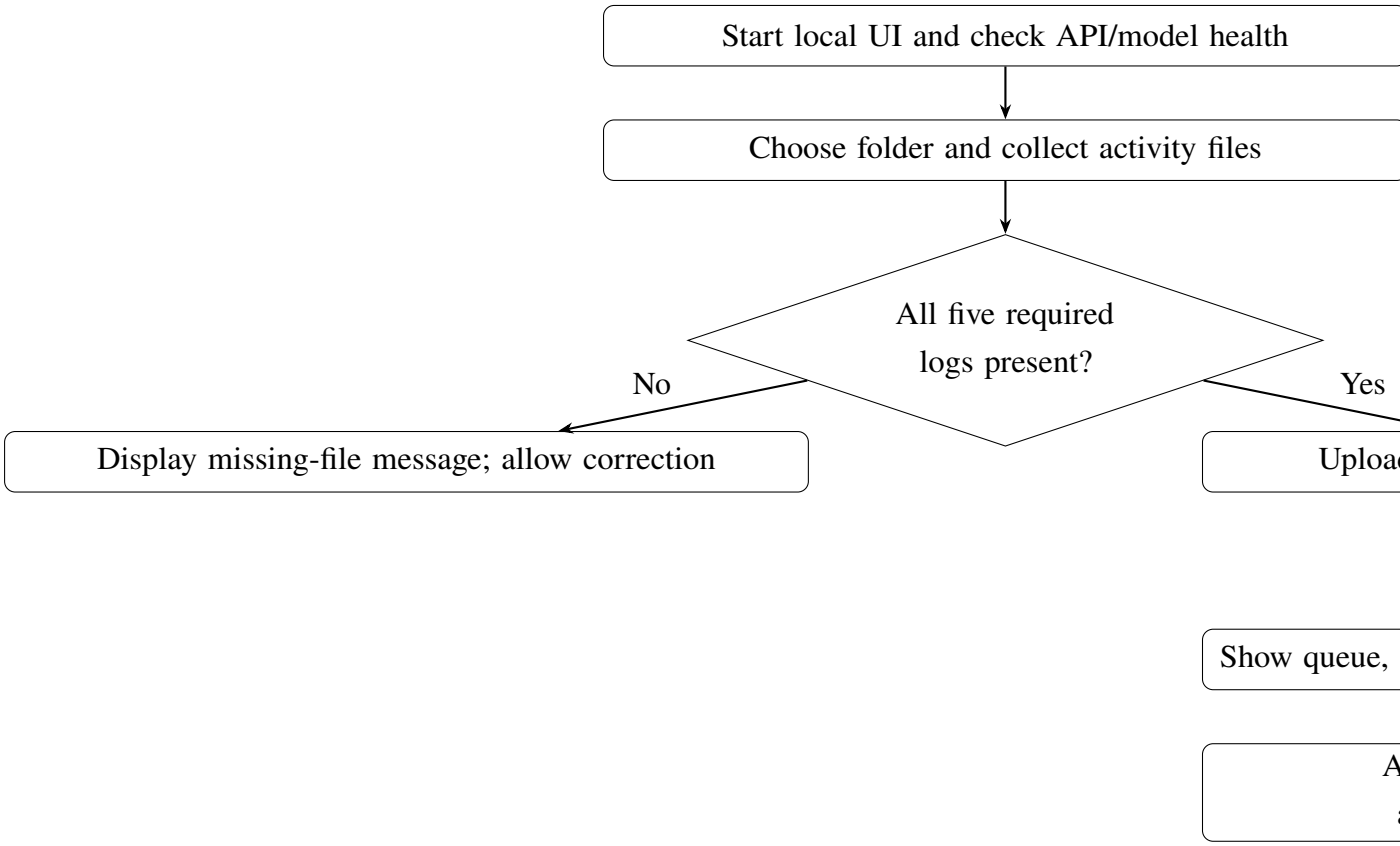


Figure 4.3: Analysis and review activity

4.5 Sequence diagram

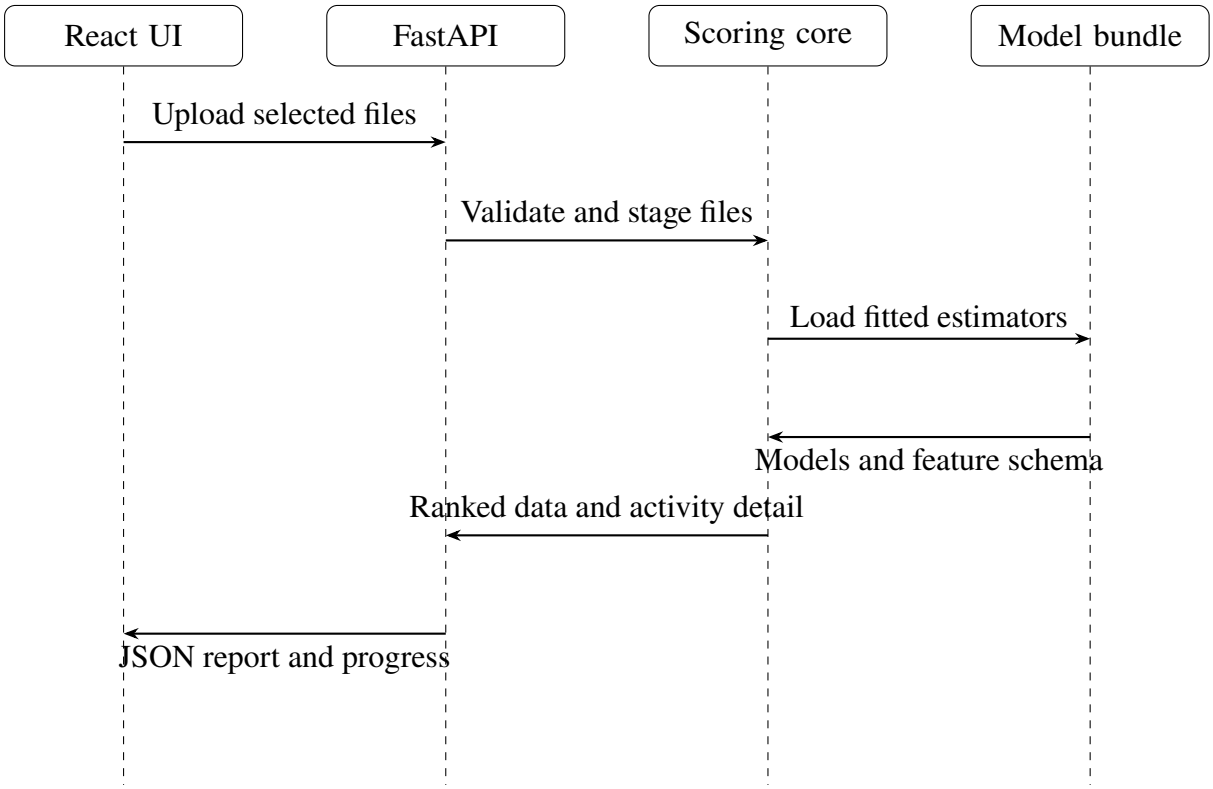


Figure 4.4: High-level sequence for a folder analysis

4.6 Data design

The project uses file-based input and output rather than a relational database. The main logical records are source events, employee-day aggregates, model scores, employee summaries, and optional display names. A user-day key joins the source aggregates. The user key joins day rows to an employee summary.

Table 4.1: Logical data entities

Entity	Key fields	Purpose
Source event	user, timestamp, source-specific fields	Original observed event; retained in input files.
Employee-day row	user, day, feature columns	Aggregated evidence used by the estimators.
Model score row	user, day, four model ranks, consensus	Relative model ranking for one employee-day.
Employee summary	user, active days, maxima, agreement, status	Review queue record and explanatory summary.
Name lookup	user, name	Optional display-only lookup; not used as model input.

The service stores the last successful analysis in process memory for employee-detail requests. The in-memory state is not durable, is shared only within the running process, and is replaced by the next analysis. The CSV exports are convenient report artifacts but do not include a transactional case history. A production design should define stable identifiers, access permissions, retention periods, integrity checks, backup policy, and an append-only audit trail.

4.7 API design

Table 4.2: Local API endpoints

Method and route	Purpose	Result and error behaviour	
`GET /api/health`	Report model-bundle readiness.	Returns a readiness Boolean and explanatory message.	
`GET /api/progress`	Read current analysis stage.	Returns percentage, stage text, and active state.	
`POST /api/analyze`	Analyse an existing local folder path.	Returns employee records and summary; invalid folders or scoring failures return errors.	
`POST /api/analyze-upload`	Receive selected activity files.	Validates required names, rejects duplicates, stages files temporarily, and returns ranked results.	
`POST /api/employee`	Request detail for one user.	Returns model scores and selected daily activity; returns conflict before analysis or not-found for unknown users.	

The browser uses the upload route so the backend can operate on a temporary staged folder. File names are reduced to their basename and compared with an allowlist of required and optional names. This is a useful input constraint, but it is not a complete security boundary: the application has no user authentication, rate limiting, CSRF protection, or production-grade isolation. It is documented to run on the loopback interface and should not be exposed to an untrusted network.

4.8 UI design

The React interface uses a dark sidebar, a light review workspace, a folder-selection card, an expandable model guide, and an analysis progress panel. After scoring, the dashboard presents summary counts, flagged employees, a searchable directory, and employee activity details. Status language is deliberately framed as review priority.

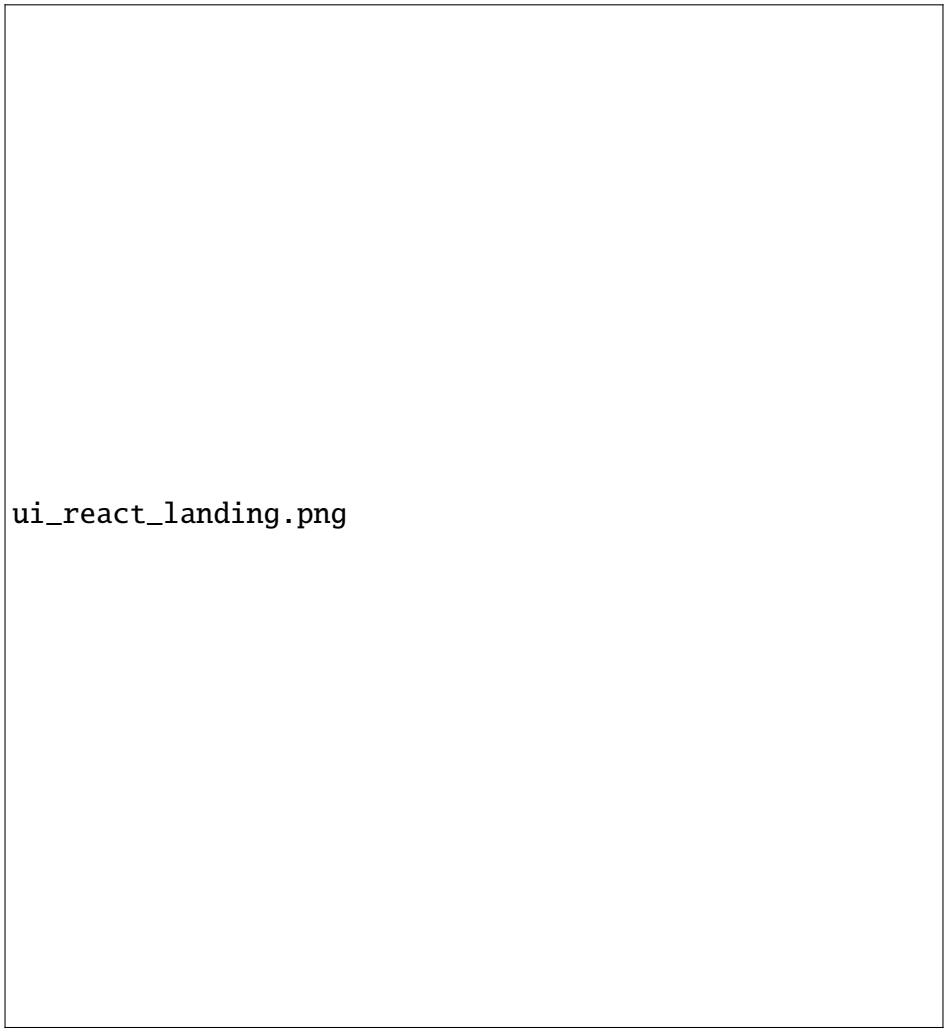


Figure 4.5: Forensio React dashboard landing state

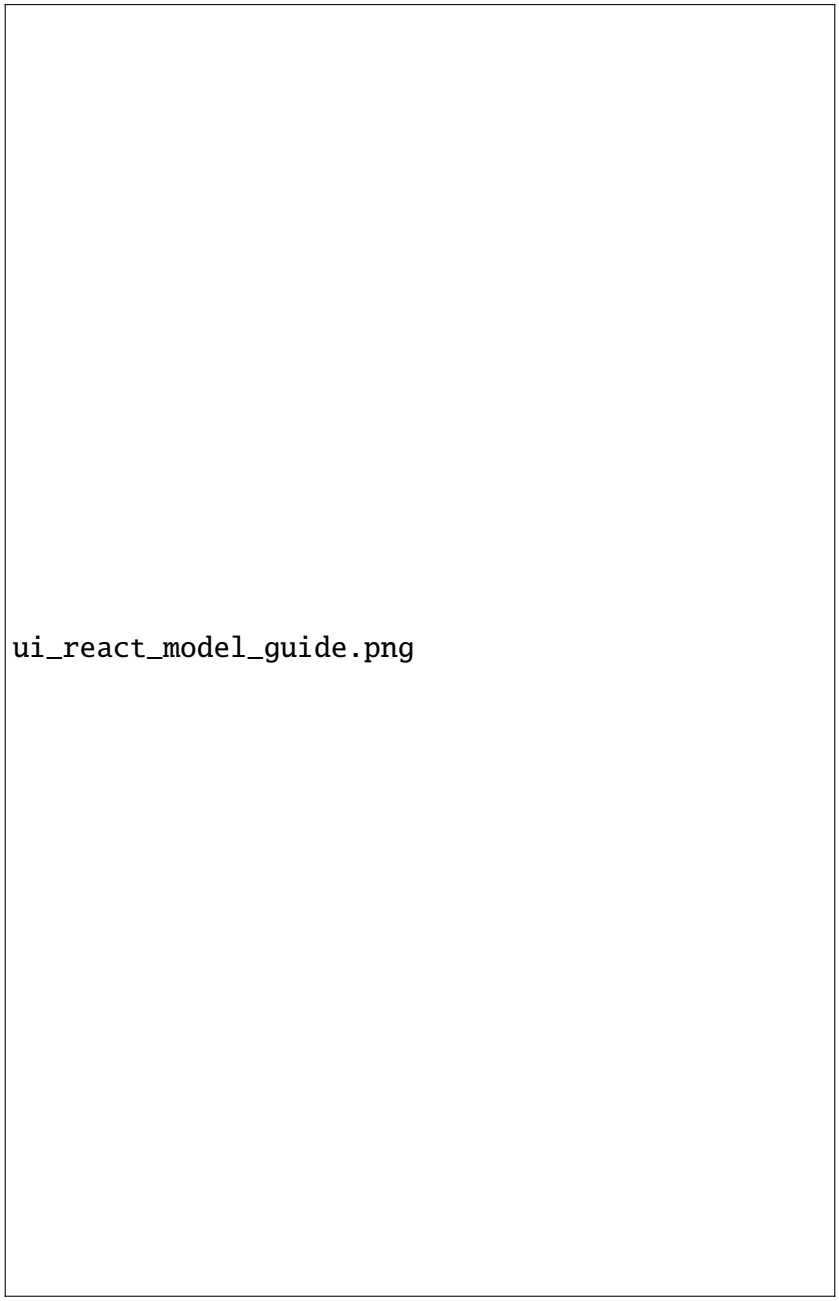


Figure 4.6: Expanded model guide and scoring explanation

The landing screenshot illustrates the initial local-analysis state. The model-guide screenshot describes the four methods and the percentile-based dashboard rule. Captures are snapshots of the interface and may not show every interaction state or the latest visual change. The report does not treat a screenshot as proof that every workflow has been tested end to end.

4.8.1 Interaction states

Table 4.3: User-interface states and visible response

State	Trigger	Expected interface response
Checking service	Page load or API not reachable.	Show availability message and retry the health request.
Ready	Model bundle is present and API is reachable.	Enable folder selection and analysis when required inputs are found.
Incomplete folder	One or more required CSVs are absent.	List the missing filenames and disable analysis.
Uploading	Browser sends selected files.	Display upload progress and disable repeated submission.
Scoring	API aggregates and scores records.	Poll progress and show current stage and estimated remaining time.
Successful analysis	API returns a report.	Render summary, flagged queue, searchable directory, and details.
Failed analysis	Service or parser returns an error.	Show an error message so the operator can correct input or inspect service logs.

4.8.2 Accessibility and usability

The React implementation uses native form controls, descriptive labels for the directory picker, a live status region for analysis progress, and a progress bar with current and maximum values. Search and status filters reduce navigation effort when the employee list is large. A textual model guide explains the distinction between model families and the review rule.

Further accessibility work should include keyboard-only workflow testing, focus management after analysis, screen-reader testing of dynamic results, high-contrast verification, and clear focus indicators. The current screenshots are visual evidence of layout, not a formal accessibility conformance assessment.

5. Results and discussion

5.1 Testing approach

The repository contains saved data-quality and evaluation artifacts, but no dedicated automated test suite was identified. This report therefore distinguishes checks reflected in the source and saved outputs from tests that would still need to be run. The test cases in Appendix 10.4 are a verification plan unless an execution result is explicitly recorded. They must not be interpreted as all passing.

The strongest available quantitative evidence is the saved employee-day model comparison and confusion-count output. The saved training notebook fixes random seeds for splitting and estimators; however, reproducibility also depends on package versions, data-file integrity, notebook execution order, and model artifact provenance.

5.2 Model planning and evaluation protocol

The labelled table contains 330,452 employee-days and 1,364 positives. The held-out test partition has 66,118 employee-days, including 289 positives. The selected queue size is 662 rows at one percent. The one-percent capacity was selected on validation data only by F1 search; the estimator variant was selected on validation Average Precision. The test split was reserved for reporting.

Evaluation is made at the employee-day level. The test confusion matrix describes whether a row is inside a known incident interval under the answer-file label rule. It does not state whether a person committed misconduct, nor does it establish the correctness of an investigative outcome.

5.3 Comparison of models

Table 5.1: Saved model comparison at the validation-selected one-percent queue capacity

Model	**Val. AP**	**Test P@1%**	**Test R@1%**	**Test F1**	**Test AP**
Random Forest	0.275	0.144	0.329	0.200	0.158
Decision Tree	0.185	0.121	0.277	0.168	0.104
Logistic Regression	0.027	0.003	0.007	0.004	0.027
Isolation Forest	0.016	0.027	0.062	0.038	0.027

At the saved one-percent capacity, Random Forest has the highest test Average Precision among the four listed models and the highest test precision, recall, and F1 at that fixed capacity. This is an observation about the saved partition and configuration only. The Decision Tree also ranks some known positive days, while Logistic Regression and Isolation Forest perform less strongly at the one-percent operating point. The results do not establish that a Random Forest is universally preferable for insider-risk work.

Table 5.2: Random Forest confusion matrix at the one-percent test queue

	Predicted not queued	**Predicted queued**
Actual negative label	65,262	567
Actual positive label	194	95

The one-percent queue contains 95 known positive employee-days and 567 rows that do not match a known positive label. It misses 194 of the 289 positive rows. These false-positive and false-negative terms are defined relative to the synthetic incident-window label, not to confirmed real-world outcomes. A row labelled negative may still require attention for reasons outside the benchmark's labels.

Table 5.3: Random Forest comparison at a five-percent capacity reference

Metric	**One percent**	**Five percent**	**Interpretation**
Queue size	662	3,306	Number of test employee-days selected.
Known positives retrieved	95	178	Test positive labels inside the queue.
Precision	0.144	0.054	Positive labels per selected record.
Recall	0.329	0.616	Positive labels found out of 289.
F1	0.200	0.099	Harmonic mean of precision and recall at each capacity.

Increasing the capacity from one to five percent retrieves more known positive days but also adds more records to the analyst queue. The one-percent setting has higher precision and F1 in the saved test result, while five percent has higher recall. This trade-off is expected in a ranked review process and should be selected according to real operational capacity and the consequences of missed leads.

5.4 Evaluation metrics

Let P be the set of test employee-days labelled positive and Q_k the k highest-scored rows. At a fixed queue size:

$$\text{Precision@ } k = \frac{|Q_k \cap P|}{k}, \qquad \text{Recall@ } k = \frac{|Q_k \cap P|}{|P|}.$$

Precision measures the positive-label fraction among selected rows. Recall measures the positive-label fraction retrieved. F1 is the harmonic mean of precision and recall. Average Precision summarizes ranking quality over score thresholds and does not depend on a single fixed queue cutoff.

Accuracy is the fraction of all records correctly classified at a chosen threshold. With 289 positive rows among 66,118 test rows, a model that predicts almost everything negative can attain high accuracy while retrieving few positives. Accuracy must therefore be read alongside positive-class metrics and queue size.

5.5 Dashboard consensus and offline evaluation

For user u, day d, and model m, the dashboard calculates a percentile rank $r_{u,d,m}$. It computes

$$c_{u,d} = \frac{1}{4} \sum_{m=1}^{4} r_{u,d,m}, \qquad C_u = \max_d c_{u,d}.$$

The application also records the number of models for which an employee's maximum score is at or above that model's employee-level 90th percentile. A user is marked `SUSPICIOUS` if the consensus rank falls within the top five percent or the agreement count is at least two; otherwise the display status is `REVIEW`.

These employee-level labels and the offline employee-day test metrics are different products. The dashboard percentile ranks depend on the selected data batch. Its top-five-percent threshold is a relative queue rule, not a calibrated threshold. The offline 1-percent test-capacity results do not validate the dashboard's top-five-percent employee rule or its reason text.

5.6 Confusion matrix interpretation

The term false positive in a benchmark confusion matrix means an employee-day selected by the model that does not overlap an incident interval in the answer file. It does not mean the activity is known to be harmless. Likewise, a false negative is an incident-window day not selected at the chosen capacity; it does not prove that the model missed a specific harmful act. These definitions matter in a high-impact setting because the labels are incomplete and synthetic.

An analyst should use a prioritized record to locate the corresponding source events,

compare them with work context and approved policy, and document alternative explanations. The model score alone should not be used as the basis for employment action, access revocation, or a claim of intent.

5.7 Discussion

The saved results show that the chosen Random Forest configuration provides the strongest ranking among the compared models on the held out synthetic test partition at the reported capacity. Synthetic augmentation was retained because validation Average Precision improved over the real-data baseline; the saved report records 815 synthetic rows and test Average Precision of 0.158 for the selected model, compared with 0.088 for the baseline. This demonstrates the value of comparing augmentation on validation data rather than assuming it helps.

The resulting precision of approximately 14.4 percent still means that most rows in the one-percent queue are not positive under the benchmark label. The remaining queue entries should not be called false alarms about real people; their labels simply do not match a listed incident window. The recall of approximately 32.9 percent also means that the majority of known positive days fall outside that capacity-limited queue. A real organisation would need to consider the cost of both unnecessary investigation and missed risk.

There are several additional limits. The test is one partition of one synthetic release; feature engineering is based on daily volume; scheduled work context is absent; and the evaluation target is a window-overlap proxy. The dashboard rule is not itself evaluated against the test labels. The screenshots document visual layout but not successful end-to-end use with an arbitrary new data folder. These factors make the system suitable for demonstration and further study, not for unreviewed operational decisions.

5.8 Requirements traceability

Table 5.4: Traceability from project goals to implementation evidence

Goal	Implementation	Evidence	Status
Aggregate five sources	Shared employee-day feature pipeline.	Core code and cleaning summary.	Implemented; cross-chunk duplicate limits documented.
Compare models	Four model families with grouped splits.	Notebook and saved comparison CSV.	Implemented for CERT r4.2.
Rank review leads	Percentile consensus and employee status rule.	Core scoring logic and dashboard.	Implemented; dashboard threshold not separately evaluated.
Support local analysis	FastAPI upload path and React client.	API routes, UI screenshots, project README.	Implemented for local demonstration.
Protect data responsibly	Local processing and omission of content features.	Design choices and report guidance.	Partial; security and governance controls remain necessary.

6. Model deployment

6.1 Local demonstration setup

The primary demonstration uses a local API and a Vite development server. The model bundle must exist at `sprint1_output/insider_investigation_model.joblib`. From the repository root, install the Python requirements if required and start the API and React application as described in the project README. The API is intended to bind to `127.0.0.1`; Vite normally serves the client at `127.0.0.1:5173`. The alternative Streamlit interface can be launched from the repository root.

Table 6.1: Demonstration startup sequence

Step	Operator action	Expected outcome
1	Activate the project's Python environment and verify the model bundle exists.	The API dependencies and four-model artifact are available.
2	Start the API on the loopback interface.	Health endpoint reports whether the bundle can be found.
3	Start the Vite client from the `frontend` directory.	Browser page loads and checks API health.
4	Select the folder containing five required activity CSV files.	UI shows selected folder and reports missing filenames if any.
5	Run analysis and follow progress.	UI shows stages while the API processes and scores records.
6	Review the summary and employee details.	Results can be examined as leads and corroborated with original evidence.

6.2 Model implementation

The API checks whether the bundle exists before analysis and loads it through the core's validation function. The core checks the expected model names and the feature list before scoring. An incomplete or missing bundle should produce an explicit error that instructs the operator to regenerate it from the notebook. The bundle is a trusted artifact and should be built, stored, and loaded only from controlled locations.

The upload endpoint accepts only an allowlist of expected source filenames and optional employee-name lookup files. It writes accepted files to a temporary directory and removes that directory after analysis. It separately writes summary CSV files into the project's output directory. The outputs contain derived records and should be handled with the same care as their

source logs.

6.3 Analyst workflow

1. Confirm that the dataset is authorised for analysis and that the selected date range and source systems match the review question.
2. Check that the folder contains all five activity files and that the files are from a consistent period and environment.
3. Run the analysis and monitor the displayed progress. If an error appears, correct the input or consult the local service log; do not treat an incomplete run as a valid report.
4. Read the summary as counts of records and relative queue status, not as a risk probability.
5. Open an employee detail and note the dates and aggregate signals that contributed to prioritisation.
6. Verify relevant original events and obtain contextual information through the organisation's approved investigation process.
7. Record evidence, alternative explanations, reviewer identity, and disposition in an authorised case-management process outside this prototype.

6.4 Operational limitations

The API retains the most recent result in process memory and serialises analysis with a process-local lock. Multiple workers would not share this state. The API writes CSV outputs but does not create a durable case database, an audit trail, or a signed report. The UI has no login, role-based permissions, or approval workflow. There is no formal monitoring for model drift, unexpected feature distributions, or queue-rate changes.

The prototype must not be exposed directly to a public network. A future deployment would require authentication, authorisation, transport protection, request size limits, secure temporary storage, input validation, audit logging, dependency management, backup and retention rules, and a documented incident response process. These controls need an architecture and threat assessment; a local demo configuration is not a substitute.

6.5 Deployment risk review

Table 6.2: Deployment risks and practical mitigations

Risk	**Potential consequence**	**Mitigation required before operational use**
Over-interpretation	Relative score treated as proof or probability.	Use clear language, training, review policy, and independent corroboration.
Unauthorised access	Activity files or exports exposed.	Authentication, least privilege, access logging, secure storage, and retention limits.
Malformed or mismatched logs	Silent feature misalignment or inaccurate ranks.	Schema validation, source/date consistency checks, and quality reports.
Untrusted model artifact	Unsafe code execution during deserialization.	Load only signed or otherwise verified artifacts from controlled storage.
Selection bias	Legitimate role or schedule differences produce unequal queue rates.	Context-aware validation, stakeholder review, and appropriate impact assessment.
Operational drift	Data or model changes degrade performance.	Monitor distributions, document versions, re-evaluate on approved representative data.

7. Git history

The repository history shows iterative development across data preparation, model evaluation, application interfaces, and report documentation. Recent commits updated the API implementation and `.gitignore`, refined report support material, and expanded project documentation. The current project contains a React dashboard, a FastAPI service, a Python scoring core, a Streamlit alternative, a modelling notebook, evaluation artifacts, and the synthetic data release.

Table 7.1: Selected repository milestones

Commit	**Date**	**Recorded milestone**
`e9bb2636`	2026-10-25	Updated ignore rules and removed a deprecated supplementary report file.
`ed6e861e`	2026-10-25	Removed deprecated API implementation details.
`1ebf6156`	2026-10-23	Added auxiliary report files and improved document structure.
`5f4ab2f7`	2026-10-23	Refactored report documentation and added supporting details.
`42d14c99`	2026-10-22	Expanded feature dictionary, lineage notes, and report sections.

The history indicates that the repository changed during report preparation. The commit list is a selected snapshot, not a complete software-development process assessment. A production project should also use reviewed pull requests, versioned model artifacts, dependency locks, automated tests, and release tags. The report source in this file is a new report document; it does not restore or overwrite the older report source that may exist in repository history.

8. Conclusions

8.1 Summary of the project

This project developed a local-first prototype for turning several employee activity logs into a review queue. The data pipeline cleans and aggregates events into employee-day counts. A notebook labels synthetic incident-window records, creates user-disjoint development and test partitions, compares four model families, selects a candidate and queue capacity on validation data, and persists a model bundle. The API and dashboard use the saved feature contract to score a new batch and expose aggregated details to an analyst.

The saved evaluation supports a limited empirical conclusion: Random Forest achieved the strongest listed performance among the four models at the selected one-percent test queue capacity on this CERT r4.2 partition. The result is not a general guarantee and should not be interpreted as detection accuracy for a real organisation. The evaluation's positive class is defined by overlap with synthetic incident windows; both false positives and false negatives are benchmark-relative.

8.2 Learning outcomes

The implementation provided experience in dataset inspection, chunked processing, feature engineering, rare-class evaluation, grouped cross-validation, model persistence, API design, browser interaction, visual communication, and responsible-use analysis. It also demonstrated the importance of keeping a training contract consistent with inference code and separating validation selection from final test reporting.

8.3 Final conclusion

Forensio demonstrates how a carefully bounded analytics pipeline can help organise the first stage of a review. Its most important design property is not the production of a score, but the explicit separation between statistical prioritisation and human judgment. The current prototype needs additional software, privacy, security, validation, and operational controls before any use beyond controlled research and demonstration.

9. Future scope

Future work should be driven by evidence and governance needs rather than by increasing model complexity alone. The following roadmap describes improvements that would make the system more reliable, interpretable, and useful while preserving the human-review boundary.

Table 9.1: Prioritised future work

Phase	Work item	Purpose and acceptance evidence	Priority
1	Add automated unit and integration tests.	Validate parsing, feature alignment, score ordering, API errors, upload staging, and status logic on controlled fixtures.	High
1	Add input schema and quality reports.	Report row counts, invalid fields, duplicate handling, missing sources, and date coverage before scoring.	High
1	Pin dependencies and version model bundles.	Reproduce training and inference with documented library versions, data checksums, and metadata.	High
2	Add model explanation and analyst feedback.	Show feature contributions cautiously and record review outcomes without treating feedback as an automatic label.	Medium
2	Evaluate temporal and cross-release generalisation.	Test robustness across time windows and benchmark versions using predefined protocols.	High
2	Add role and schedule context under governance.	Reduce avoidable confounding only when lawful, necessary, validated, and stakeholder-approved.	Medium
3	Build case-management and audit integration.	Provide authorised review history, reviewer notes, evidence references, and retention controls.	Medium
3	Assess privacy and security controls.	Complete threat modelling, privacy impact review, access control, and deployment testing.	High

9.1 Model and evaluation improvements

Model work should first establish stronger evaluation discipline. Experiments should compare the current random grouped split with time-aware and release-aware designs, preserve an untouched test set, and report uncertainty intervals where the sampling design supports them. Synthetic augmentation should be compared with unaugmented baselines through repeated validation, with data-generation assumptions documented. Performance should be reported at several review capacities and accompanied by calibration analysis only if a probability interpretation is intended.

The dashboard's employee-level consensus rule requires its own evaluation. Candidate thresholds should be chosen on validation data, analyst capacity should be specified before test evaluation, and employee-level labels should be aggregated under an explicit rule. Evaluation should also examine how queue membership changes between batches and whether changes are caused by data composition rather than model behaviour.

9.2 Software and governance improvements

Engineering priorities include a locked environment, automated regression tests, structured logs, configurable input limits, secure temporary storage, consistent error handling, and a versioned model-bundle manifest. The web service should receive a threat model before any network exposure. A production workflow should add authentication, role permissions, protected exports, durable audit events, and a documented retention schedule.

Governance work should define the permitted purpose, data sources, access rights, legal basis, review standards, appeal or correction process, and criteria for ending a review. Any use involving real people should undergo appropriate institutional and legal review, impact assessment, and consultation with relevant stakeholders. A human reviewer must be able to reject a model lead and record a reasoned disposition.

10. Appendix

10.1 Minimum software requirements

Table 10.1: Software needed for the current demonstration

Software	**Purpose**	**Project note**
Python 3	Model scoring, API, and alternative Streamlit UI.	Use an environment compatible with the installed scientific packages.
Python dependencies	pandas, NumPy, scikit-learn, joblib, FastAPI, python-multipart, Streamlit, Uvicorn.	Install from the repository requirements file and record resolved versions.
Node.js and npm	React and Vite development client.	Use the frontend package manifest and lock file when available.
Modern browser	Select local activity files and render the dashboard.	Directory upload support may differ by browser.
LaTeX distribution	Compile this report with pdfLaTeX.	The report uses standard packages including TikZ and newtx.
Model artifact	Persisted estimator bundle.	Expected path is `sprint1_output/insider_investigation_model.jo`

10.2 Minimum hardware requirements

The repository does not contain a measured minimum hardware benchmark. Actual resource use depends on input size, concurrent processes, Python package versions, and the fitted estimators. The following table is a practical starting point for a student demonstration and must not be treated as a tested guarantee.

Table 10.2: Practical workstation guidance, not a measured benchmark

Resource	Suggested starting point	Reason
Processor	Modern multi-core desktop or laptop CPU.	CSV parsing and model scoring benefit from general-purpose compute; exact runtime is unmeasured.
Memory	16 GB RAM recommended for the full synthetic dataset.	Chunked input lowers read size, but merged employee-day tables and models remain in memory.
Storage	Several gigabytes of free space for source logs, environments, outputs, and report assets.	Dataset copies, notebook artifacts, and dependencies occupy disk space.
Display	1366 by 768 pixels or higher.	Gives reasonable room for the dashboard and report figures.
Network	Not required for local analysis after dependencies are installed.	The documented demo binds locally; package installation may require connectivity.

10.3 Input contract and data lineage

Table 10.3: Input contract checklist

Check	Expected condition	Operator action
Required files	All five expected filenames are present.	Confirm that the selected folder is the intended dataset.
Date and user	Each input contains parsable date and user fields.	Correct the source export; do not silently substitute another identity column.
Time coverage	Sources cover a consistent period and time zone.	Review date ranges and document any intentional gaps.
Duplicates	Cleaning results are understood.	Consider whether exact global deduplication is required.
Optional names	Lookup has a recognised user and name column.	Treat display names as sensitive and keep them out of model features.
Model schema	Bundle feature list matches the intended model release.	Verify bundle provenance and avoid untrusted serialized files.

10.4 Test cases and verification plan

Table 10.4: Functional and data-quality test plan

ID	Scenario	Expected result	Execution status
T-01	Valid folder with five source files and model bundle.	Analysis returns employee summaries and activity rows.	Not recorded as executed in this report.
T-02	Missing one required CSV.	API reports the missing filename; no successful result is returned.	Not recorded as executed in this report.
T-03	Duplicate accepted filename in uploaded folder.	API returns a clear duplicate-file error.	Not recorded as executed in this report.
T-04	CSV missing date or user column.	Scorer raises a schema error identifying the source.	Not recorded as executed in this report.
T-05	Invalid date and blank user rows.	Invalid rows do not contribute to daily aggregates.	Cleaning summaries show no such rows for saved dataset; synthetic fixture test not recorded.
T-06	Model bundle lacks a required model or feature list.	Bundle validation raises an actionable error.	Not recorded as executed in this report.
T-07	Required model feature absent from aggregate.	Inference inserts a zero column in the saved feature order.	Behaviour present in source; automated test not recorded.
T-08	Request employee detail before an analysis.	API returns conflict status and explanatory detail.	Not recorded as executed in this report.
T-09	Request an unknown employee after analysis.	API returns not-found status.	Not recorded as executed in this report.
T-10	API unavailable when UI loads.	UI communicates local service unavailability and retries health.	Interaction visible in source; browser test not recorded.

Continued on next page

ID	Scenario	Expected result	Execution status
T-11	Run analysis with a large CSV.	Chunk processing completes within tested memory and time bounds.	Benchmark not recorded; measure before operational use.
T-12	Repeat analysis with the same verified inputs and bundle.	Outputs are reproducible within expected numeric tolerance.	End-to-end repeatability test not recorded.

10.5 Evaluation metrics and saved counts

Table 10.5: Saved one-percent test confusion counts by model

Model	TN	FP	FN	TP
Logistic Regression	65,169	660	287	2
Decision Tree	65,247	582	209	80
Random Forest	65,262	567	194	95
Isolation Forest	65,185	644	271	18

The confusion counts are useful for reconstructing capacity metrics. For Random Forest, $TP + FP = 662$, equal to the selected one-percent queue size. There are $TP + FN = 289$ positive test labels. This yields precision $95/662$, recall $95/289$, and the corresponding F1. Small differences in displayed values arise from rounding.

10.6 Troubleshooting guide

Table 10.6: Common demonstration issues

Symptom	Likely cause	Recommended check
Health reports model missing	Bundle absent or wrong path.	Run the notebook model-bundle creation step and verify the expected filename.
Browser cannot reach the API	API is stopped or listening on a different address.	Start the API using the documented local command and inspect its terminal.
Analysis says files are missing	Folder selection omitted a source or names differ.	Confirm all five exact filenames are present in the selected folder.
Scoring fails on a CSV	Unsupported schema, encoding, or malformed content.	Check headers, delimiter, date values, file integrity, and service error detail.
No employee details appear	Analysis did not complete or selected user is absent.	Re-run analysis, then select a user returned in the current report.
Results differ from prior run	Input batch, model bundle, package version, or row order differs.	Record data and artifact hashes, environment versions, and run settings.

10.7 Responsible-use checklist

Before any review involving real people, an accountable organisation should verify that all of the following are satisfied:

- The review purpose, authority, data sources, and data period are documented and approved.
- Access to raw logs, derived reports, and employee-name lookups is limited to authorised personnel.
- The model's limitations and relative score meaning are explained to each reviewer.
- A score cannot, by itself, trigger disciplinary, employment, access, or law-enforcement action.
- Reviewers can inspect original records, record alternative explanations, and correct erroneous identity or source data.
- Retention, deletion, disclosure, and audit requirements are defined before data is loaded.

- The model and thresholds have been assessed for the intended population and operating environment.
- The organisation has a process to pause the tool when data quality, access control, or model behaviour is unacceptable.

10.8 Glossary

Table 10.7: Terms used in the report

Term	**Meaning in this project**
Activity event	One row in one of the input CSV source logs.
Employee-day	Aggregated feature record for one user identifier on one calendar day.
Incident-window label	Positive label when a user-day overlaps a listed synthetic incident interval.
Review capacity	Fraction or count of highest-ranked rows selected for human review.
Average Precision	Ranking metric summarising precision across recall changes as the score threshold varies.
Percentile rank	Relative position of a score within the current scored batch.
Model agreement	Count of model employee-level ranks that meet the per-model top-ten-percent rule.
False positive	Selected test record without a matching positive label under the benchmark rule.
False negative	Positive benchmark record not selected at the chosen queue capacity.
Data lineage	Trace from raw source records through cleaning and features to scores and outputs.

10.9 Reproducibility checklist

1. Record the repository commit, Python version, dependency versions, and operating system.
2. Verify the dataset release, input filenames, checksums, and licensing conditions.
3. Re-run the notebook from a clean environment and confirm cell execution order.
4. Confirm the user-group split has no overlap and that each partition contains positive examples.

5. Save the training and validation selection outputs, model parameters, feature list, and augmentation decision.
6. Keep the test metrics separate from all selection decisions.
7. Record the model-bundle hash and the source logs used for each dashboard run.
8. Compile this document twice and inspect page count, contents, tables, figures, code listings, and page-number transitions.

10.10 Notebook source listing

The following appendix listing is generated from the project's notebook code cells. It is included as a reproducibility aid and should be refreshed from `notebook/forensic.ipynb` when the notebook changes. It is source documentation, not a substitute for executing the notebook and reviewing its outputs.

```
# Notebook code cell 2
from pathlib import Path
import json

import pandas as pd
import matplotlib.pyplot as plt

# Notebook code cell 4
project_dir = Path.cwd()

if project_dir.name == "notebook":
    project_dir = project_dir.parent

data_dir = project_dir / "r4.2"
output_dir = project_dir / "sprint1_output"
output_dir.mkdir(exist_ok=True)

required_files = [
    "logon.csv",
    "device.csv",
    "file.csv",
    "email.csv",
    "http.csv",
]

print("Project folder:", project_dir)
print("Data folder:", data_dir)
print("Output folder:", output_dir)

# Notebook code cell 5
missing_files = [
    name for name in required_files
    if not (data_dir / name).is_file()
]

if missing_files:
    print("These files are missing:", missing_files)
else:
    print("All five activity files are present.")

# Notebook code cell 7
for filename in required_files:
    file_path = data_dir / filename
    sample = pd.read_csv(file_path, nrows=5)

    print(f"\n{filename}")
    print("Columns:", list(sample.columns))
    display(sample.head(3))

# Notebook code cell 9
chunk_size = 25_000

def process_activity_file(filename):
    file_path = data_dir / filename
    source = file_path.stem.lower()

    # Read the header first so we can select only useful columns.
    available_columns = pd.read_csv(file_path, nrows=0).columns.tolist()
```

```
    available_columns = [column.strip().lower() for column in available_columns]

    useful_columns = ["date", "user", "pc", "activity", "attachments", "filename"]
    columns_to_read = [
        column for column in useful_columns
        if column in available_columns
    ]

    if "date" not in columns_to_read or "user" not in columns_to_read:
        raise ValueError(f"{filename} needs both 'date' and 'user' columns.")

    daily_parts = []
    quality = {
        "rows_read": 0,
        "rows_kept": 0,
        "missing_user": 0,
        "invalid_date": 0,
        "duplicates_removed": 0,
    }

    for chunk in pd.read_csv(
        file_path,
        usecols=lambda column: column.strip().lower() in columns_to_read,
        chunksize=chunk_size,
        low_memory=False,
    ):
        chunk.columns = [column.strip().lower() for column in chunk.columns]
        quality["rows_read"] += len(chunk)

        chunk["user"] = chunk["user"].astype("string").str.strip()
        missing_user = chunk["user"].isna() | chunk["user"].eq("")
        quality["missing_user"] += int(missing_user.sum())

        dates = pd.to_datetime(chunk["date"], errors="coerce", format="mixed")
        invalid_date = dates.isna()
        quality["invalid_date"] += int(invalid_date.sum())

        duplicate = chunk.duplicated()
        quality["duplicates_removed"] += int(duplicate.sum())

        keep = ~(missing_user | invalid_date | duplicate)
        chunk = chunk.loc[keep].copy()
        dates = dates.loc[keep]

        if chunk.empty:
            continue

        quality["rows_kept"] += len(chunk)

        # A row is outside normal working hours if it is before 6 AM
        # or at/after 6 PM, based on the timestamp in the dataset.
        hour = dates.dt.hour
        chunk["after_hours_count"] = (
            (hour < 6) | (hour >= 18)
        ).astype("int8").to_numpy()

        chunk["day"] = dates.dt.strftime("%Y-%m-%d").to_numpy()
        chunk["event_count"] = 1

```

```
        feature_columns = ["event_count", "after_hours_count"]

        if source == "device" and "activity" in chunk.columns:
            chunk["device_connect_count"] = (
                chunk["activity"].astype("string").str.lower() == "connect"
            ).astype("int8")
            feature_columns.append("device_connect_count")

        if source == "email" and "attachments" in chunk.columns:
            chunk["attachment_count"] = (
                pd.to_numeric(chunk["attachments"], errors="coerce")
                .fillna(0)
            )
            feature_columns.append("attachment_count")

        group = chunk.groupby(["user", "day"])[feature_columns].sum()
        group = group.rename(
            columns={column: f"{source}_{column}" for column in feature_columns}
        )
        daily_parts.append(group)

    if daily_parts:
        daily_features = (
            pd.concat(daily_parts)
            .groupby(level=["user", "day"])
            .sum()
        )
    else:
        daily_features = pd.DataFrame()

    return daily_features, quality

# Notebook code cell 10
daily_features_by_source = []
data_quality = {}

for filename in required_files:
    print(f"Processing {filename}...")
    source_features, source_quality = process_activity_file(filename)

    daily_features_by_source.append(source_features)
    data_quality[filename] = source_quality

    print(
        f"  Kept {source_quality['rows_kept']:,} "
        f"of {source_quality['rows_read']:,} rows"
    )

# Notebook code cell 12
employee_daily = pd.concat(daily_features_by_source, axis=1).fillna(0)
employee_daily = employee_daily.groupby(level=["user", "day"]).sum()
employee_daily = employee_daily.reset_index()

employee_daily.head()

# Notebook code cell 13
features_path = output_dir / "employee_daily_features.csv"
quality_path = output_dir / "data_quality_summary.json"

```

```
employee_daily.to_csv(features_path, index=False)
quality_path.write_text(
    json.dumps(data_quality, indent=2),
    encoding="utf-8",
)

print("Saved cleaned features to:", features_path)
print("Saved data-quality summary to:", quality_path)
print("Employee-day rows:", f"{len(employee_daily):,}")
print("Employees:", f"{employee_daily['user'].nunique():,}")

# Notebook code cell 15
event_columns = [
    column for column in employee_daily.columns
    if column.endswith("_event_count")
]

events_by_source = employee_daily[event_columns].sum().sort_values(
    ascending=False
)

display(events_by_source.to_frame("event_count"))

events_by_source.plot(
    kind="bar",
    figsize=(9, 5),
    title="Recorded activity by source",
    ylabel="Event count",
    xlabel="Activity source",
    color="steelblue",
)

plt.xticks(rotation=30, ha="right")
plt.tight_layout()
plt.show()

# Notebook code cell 16
employee_daily["day"] = pd.to_datetime(employee_daily["day"])

daily_event_totals = (
    employee_daily.groupby("day")[event_columns]
    .sum()
    .sum(axis=1)
)

daily_event_totals.plot(
    figsize=(12, 5),
    title="Daily activity volume",
    ylabel="Event count",
    xlabel="Date",
    color="darkorange",
)

plt.tight_layout()
plt.show()

# Notebook code cell 17
employee_event_totals = (
    employee_daily.groupby("user")[event_columns]
```

```
    .sum()
    .sum(axis=1)
    .sort_values(ascending=False)
)

top_employees = employee_event_totals.head(10).rename("event_count")
display(top_employees.to_frame())

top_employees.plot(
    kind="bar",
    figsize=(10, 5),
    title="Employees with the most recorded events",
    ylabel="Event count",
    xlabel="Employee ID",
    color="slateblue",
)

plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.show()

# Notebook code cell 20
answers_file = project_dir / "answers" / "insiders.csv"

incidents = pd.read_csv(answers_file)

incidents = incidents[incidents["dataset"].astype(str) == "4.2"].copy()

incidents["start"] = pd.to_datetime(
    incidents["start"],
    format="%m/%d/%Y %H:%M:%S",
    errors="coerce",
)

incidents["end"] = pd.to_datetime(
    incidents["end"],
    format="%m/%d/%Y %H:%M:%S",
    errors="coerce",
)

incidents = incidents.dropna(subset=["start", "end"])
incidents.head()

# Notebook code cell 21
employee_daily["day"] = pd.to_datetime(employee_daily["day"]).dt.normalize()

employee_daily["known_incident_window"] = 0
employee_daily["scenario"] = "not_listed"

for incident in incidents.itertuples(index=False):
    start_day = incident.start.normalize()
    end_day = incident.end.normalize()

    in_incident_window = (
        employee_daily["user"].eq(incident.user)
        & employee_daily["day"].between(start_day, end_day)
    )

    employee_daily.loc[in_incident_window, "known_incident_window"] = 1
```

```
    employee_daily.loc[in_incident_window, "scenario"] = str(incident.scenario)

print("Known incident cases:", len(incidents))
print("Employee-days:", len(employee_daily))
print(
    "Employee-days inside known incident windows:",
    int(employee_daily["known_incident_window"].sum()),
)

# Notebook code cell 22
label_counts = employee_daily["known_incident_window"].value_counts().sort_index()
label_counts.index = ["Not listed in incident windows", "Inside incident window"]
display(label_counts.to_frame("employee_day_count"))

labeled_file = output_dir / "employee_daily_labeled.csv"
employee_daily.to_csv(labeled_file, index=False)

print("Saved labeled employee-days to:", labeled_file)

# Notebook code cell 24
import numpy as np

from sklearn.ensemble import IsolationForest, RandomForestClassifier
from sklearn.base import clone
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import average_precision_score
from sklearn.model_selection import StratifiedGroupKFold
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.tree import DecisionTreeClassifier

# Notebook code cell 25
target_column = "known_incident_window"

# Do not let employee IDs, dates, scenarios, or the answer label become model inputs.
excluded_columns = {
    "user",
    "day",
    "scenario",
    target_column,
}

feature_columns = [
    column for column in employee_daily.select_dtypes(include="number").columns
    if column not in excluded_columns
]

X = employee_daily[feature_columns].fillna(0)
y = employee_daily[target_column].astype(int)
groups = employee_daily["user"]

print("Features:", feature_columns)
print("Employee-days:", len(employee_daily))
print("Incident-window labels:", int(y.sum()))

# Hold out employees, not individual days, to avoid the same employee appearing
# in both model development and evaluation data.
outer_splitter = StratifiedGroupKFold(
```

```
    n_splits=5,
    shuffle=True,
    random_state=42,
)

train_validation_indices, test_indices = next(
    outer_splitter.split(X, y, groups=groups)
)

inner_splitter = StratifiedGroupKFold(
    n_splits=4,
    shuffle=True,
    random_state=43,
)
train_relative_indices, validation_relative_indices = next(
    inner_splitter.split(
        X.iloc[train_validation_indices],
        y.iloc[train_validation_indices],
        groups=groups.iloc[train_validation_indices],
    )
)
train_indices = train_validation_indices[train_relative_indices]
validation_indices = train_validation_indices[validation_relative_indices]

X_train = X.iloc[train_indices]
X_validation = X.iloc[validation_indices]
X_test = X.iloc[test_indices]
y_train = y.iloc[train_indices]
y_validation = y.iloc[validation_indices]
y_test = y.iloc[test_indices]

train_users = set(groups.iloc[train_indices])
validation_users = set(groups.iloc[validation_indices])
test_users = set(groups.iloc[test_indices])

print("Training employees:", len(train_users))
print("Validation employees:", len(validation_users))
print("Test employees:", len(test_users))
print("Training positive employee-days:", int(y_train.sum()))
print("Validation positive employee-days:", int(y_validation.sum()))
print("Test positive employee-days:", int(y_test.sum()))
print(
    "Employees shared between splits:",
    len(train_users & validation_users)
    + len(train_users & test_users)
    + len(validation_users & test_users),
)

if min(y_train.sum(), y_validation.sum(), y_test.sum()) == 0:
    raise ValueError("Each grouped split must contain incident-window examples.")

# Notebook code cell 26
models = {
    "Logistic Regression": make_pipeline(
        SimpleImputer(strategy="median"),
        StandardScaler(),
        LogisticRegression(
            max_iter=1000,
            class_weight="balanced",
```

```
            random_state=42,
        ),
    ),
    "Decision Tree": DecisionTreeClassifier(
        max_depth=10,
        min_samples_leaf=10,
        class_weight="balanced",
        random_state=42,
    ),
    "Random Forest": RandomForestClassifier(
        n_estimators=200,
        min_samples_leaf=5,
        class_weight="balanced_subsample",
        n_jobs=-1,
        random_state=42,
    ),
    # This model learns what is unusual without using the incident labels.
    "Isolation Forest": IsolationForest(
        n_estimators=200,
        contamination="auto",
        random_state=42,
        n_jobs=-1,
    ),
}


def make_synthetic_positive_counts(X_source, y_source, random_state=2026):
    """Create one count-perturbed positive example per real positive training row."""
    rng = np.random.default_rng(random_state)
    values = X_source.to_numpy(dtype=float)
    positive_rows = values[np.asarray(y_source) == 1]
    if len(positive_rows) == 0:
        raise ValueError("Synthetic augmentation needs positive training examples.")

    anchors = positive_rows[rng.integers(0, len(positive_rows), size=len(positive_rows))]
    synthetic = np.zeros_like(anchors)
    scale = rng.lognormal(mean=0.0, sigma=0.12, size=len(anchors))
    columns = list(X_source.columns)

    # Resample event counts first, then preserve the source-level count relationships.
    for source in ("logon", "device", "file", "email", "http"):
        event_column = f"{source}_event_count"
        if event_column in columns:
            event_index = columns.index(event_column)
            synthetic[:, event_index] = rng.poisson(
                np.maximum(anchors[:, event_index], 0) * scale
            )

        after_hours_column = f"{source}_after_hours_count"
        if event_column in columns and after_hours_column in columns:
            after_index = columns.index(after_hours_column)
            event_index = columns.index(event_column)
            rate = np.divide(
                anchors[:, after_index],
                anchors[:, event_index],
                out=np.zeros(len(anchors), dtype=float),
                where=anchors[:, event_index] > 0,
            ).clip(0, 1)
            synthetic[:, after_index] = rng.binomial(
```

```
                synthetic[:, event_index].astype(int), rate
            )

    connect_column = "device_device_connect_count"
    device_events = "device_event_count"
    if connect_column in columns and device_events in columns:
        connect_index = columns.index(connect_column)
        device_index = columns.index(device_events)
        connect_rate = np.divide(
            anchors[:, connect_index],
            anchors[:, device_index],
            out=np.zeros(len(anchors), dtype=float),
            where=anchors[:, device_index] > 0,
        ).clip(0, 1)
        synthetic[:, connect_index] = rng.binomial(
            synthetic[:, device_index].astype(int), connect_rate
        )

    attachment_column = "email_attachment_count"
    if attachment_column in columns:
        attachment_index = columns.index(attachment_column)
        synthetic[:, attachment_index] = rng.poisson(
            np.maximum(anchors[:, attachment_index], 0) * scale
        )

    return pd.DataFrame(synthetic, columns=columns, index=None)


validation_risk_scores = {}
risk_scores = {}
selected_synthetic_augmentation = {name: False for name in models}
baseline_rf_model = None
baseline_rf_test_scores = None

for name, model in models.items():
    print(f"Training {name}...")
    if name == "Random Forest":
        # Keep a real-data baseline, then use validation AP to choose whether
        # one synthetic count-perturbed row per positive training row helps.
        baseline_rf_model = clone(model).fit(X_train, y_train)
        baseline_validation = baseline_rf_model.predict_proba(X_validation)[:, 1]
        synthetic_X = make_synthetic_positive_counts(X_train, y_train)
        augmented_X = pd.concat([X_train.reset_index(drop=True), synthetic_X], ignore_index=True)
        augmented_y = pd.concat(
            [y_train.reset_index(drop=True), pd.Series(np.ones(len(synthetic_X), dtype=int))],
            ignore_index=True,
        )
        augmented_model = clone(model).fit(augmented_X, augmented_y)
        augmented_validation = augmented_model.predict_proba(X_validation)[:, 1]
        if average_precision_score(y_validation, augmented_validation) > average_precision_score(
            y_validation, baseline_validation
        ):
            model = augmented_model
            selected_synthetic_augmentation[name] = True
            print(f"  Validation selected synthetic augmentation ({len(synthetic_X):,} generated rows).")
        else:
            model = baseline_rf_model
            print("  Validation retained the real-data baseline.")
        baseline_rf_test_scores = baseline_rf_model.predict_proba(X_test)[:, 1]
```

```
    elif name == "Isolation Forest":
        model.fit(X_train)
    else:
        model.fit(X_train, y_train)

    models[name] = model
    if name == "Isolation Forest":
        validation_risk_scores[name] = -model.decision_function(X_validation)
        risk_scores[name] = -model.decision_function(X_test)
    else:
        validation_risk_scores[name] = model.predict_proba(X_validation)[:, 1]
        risk_scores[name] = model.predict_proba(X_test)[:, 1]

print("Model variants were selected using validation data only.")


# Notebook code cell 28
from sklearn.metrics import (
    accuracy_score,
    average_precision_score,
    confusion_matrix,
    precision_score,
    recall_score,
    f1_score,
    ConfusionMatrixDisplay,
)

# Choose a practical review capacity on validation data only. Selecting from a
# coarse grid is less sensitive to one exact cutoff than searching every row.
candidate_review_rates = [0.01, 0.02, 0.03, 0.05, 0.075, 0.10, 0.15, 0.20]
validation_labels = y_validation.to_numpy()
actual_labels = y_test.to_numpy()

# Preserve model selection by validation Average Precision.
selected_model_for_rate = max(
    validation_risk_scores,
    key=lambda name: average_precision_score(
        validation_labels, np.asarray(validation_risk_scores[name])
    ),
)
selected_model_validation_scores = np.asarray(
    validation_risk_scores[selected_model_for_rate]
)

best_rate = None
best_validation_f1 = -1.0
for candidate_rate in candidate_review_rates:
    candidate_count = max(
        1, int(np.ceil(len(validation_labels) * candidate_rate))
    )
    candidate_indices = np.argsort(selected_model_validation_scores)[-candidate_count:]
    candidate_predictions = np.zeros(len(validation_labels), dtype=int)
    candidate_predictions[candidate_indices] = 1
    candidate_f1 = f1_score(
        validation_labels, candidate_predictions, zero_division=0
    )
    # Rates are ascending, so retaining the first tie favors the smaller queue.
    if candidate_f1 > best_validation_f1:
        best_validation_f1 = candidate_f1
```

```
        best_rate = candidate_rate

selected_queue_rate = float(best_rate)
validation_top_count = max(
    1, int(np.ceil(len(validation_labels) * selected_queue_rate))
)
test_top_count = max(1, int(np.ceil(len(actual_labels) * selected_queue_rate)))
validation_five_percent_count = max(
    1, int(np.ceil(len(validation_labels) * 0.05))
)
test_five_percent_count = max(1, int(np.ceil(len(actual_labels) * 0.05)))

print(
    f"Selected model: {selected_model_for_rate}; "
    f"validation-F1 queue capacity: {selected_queue_rate:.1%} "
    f"({validation_top_count:,} of {len(validation_labels):,} employee-days); "
    f"validation F1: {best_validation_f1:.3f}"
)

results = []
confusion_matrices = {}
for name, test_scores in risk_scores.items():
    validation_scores = np.asarray(validation_risk_scores[name])
    test_scores = np.asarray(test_scores)

    validation_top_indices = np.argsort(validation_scores)[-validation_top_count:]
    validation_predictions = np.zeros(len(validation_labels), dtype=int)
    validation_predictions[validation_top_indices] = 1
    test_top_indices = np.argsort(test_scores)[-test_top_count:]
    test_predictions = np.zeros(len(actual_labels), dtype=int)
    test_predictions[test_top_indices] = 1

    # Keep the original 5% capacity as a reference for the precision/recall tradeoff.
    validation_five_indices = np.argsort(validation_scores)[-validation_five_percent_count:]
    validation_five_predictions = np.zeros(len(validation_labels), dtype=int)
    validation_five_predictions[validation_five_indices] = 1
    test_five_indices = np.argsort(test_scores)[-test_five_percent_count:]
    test_five_predictions = np.zeros(len(actual_labels), dtype=int)
    test_five_predictions[test_five_indices] = 1

    test_matrix = confusion_matrix(
        actual_labels, test_predictions, labels=[0, 1]
    )
    confusion_matrices[name] = test_matrix

    baseline_rf_top_indices = np.argsort(baseline_rf_test_scores)[-test_top_count:]
    baseline_rf_predictions = np.zeros(len(actual_labels), dtype=int)
    baseline_rf_predictions[baseline_rf_top_indices] = 1

    results.append({
        "Model": name,
        "Selected queue rate": selected_queue_rate,
        "Validation AP": average_precision_score(
            validation_labels, validation_scores
        ),
        "Validation precision @ selected queue": precision_score(
            validation_labels, validation_predictions, zero_division=0
        ),
        "Validation recall @ selected queue": recall_score(
```

```
            validation_labels, validation_predictions, zero_division=0
        ),
        "Validation F1 @ selected queue": f1_score(
            validation_labels, validation_predictions, zero_division=0
        ),
        "Test precision @ selected queue": precision_score(
            actual_labels, test_predictions, zero_division=0
        ),
        "Test recall @ selected queue": recall_score(
            actual_labels, test_predictions, zero_division=0
        ),
        "Test F1 @ selected queue": f1_score(
            actual_labels, test_predictions, zero_division=0
        ),
        "Test false alerts @ selected queue": int(test_matrix[0, 1]),
        "Test AP": average_precision_score(actual_labels, test_scores),
        "Validation precision @ 5%": precision_score(
            validation_labels, validation_five_predictions, zero_division=0
        ),
        "Validation recall @ 5%": recall_score(
            validation_labels, validation_five_predictions, zero_division=0
        ),
        "Test precision @ 5%": precision_score(
            actual_labels, test_five_predictions, zero_division=0
        ),
        "Test recall @ 5%": recall_score(
            actual_labels, test_five_predictions, zero_division=0
        ),
        "Test accuracy @ 5%": accuracy_score(actual_labels, test_five_predictions),
        "Test F1 @ 5%": f1_score(
            actual_labels, test_five_predictions, zero_division=0
        ),
        "Synthetic rows added": (
            int(y_train.sum()) if name == "Random Forest" and selected_synthetic_augmentation[name] else 0
        ),
        "RF baseline Test AP": (
            average_precision_score(actual_labels, baseline_rf_test_scores)
            if name == "Random Forest" else np.nan
        ),
        "RF baseline Test precision @ selected queue": (
            precision_score(actual_labels, baseline_rf_predictions, zero_division=0)
            if name == "Random Forest" else np.nan
        ),
        "RF baseline Test recall @ selected queue": (
            recall_score(actual_labels, baseline_rf_predictions, zero_division=0)
            if name == "Random Forest" else np.nan
        ),
        "RF baseline Test F1 @ selected queue": (
            f1_score(actual_labels, baseline_rf_predictions, zero_division=0)
            if name == "Random Forest" else np.nan
        ),
    })

# Select the model using validation AP; never use test metrics for selection.
model_comparison = (
    pd.DataFrame(results)
    .sort_values("Validation AP", ascending=False)
    .reset_index(drop=True)
)
```

```
display(model_comparison.style.format({
    "Selected queue rate": "{:.1%}",
    "Validation AP": "{:.3f}",
    "Validation precision @ selected queue": "{:.3f}",
    "Validation recall @ selected queue": "{:.3f}",
    "Validation F1 @ selected queue": "{:.3f}",
    "Test precision @ selected queue": "{:.3f}",
    "Test recall @ selected queue": "{:.3f}",
    "Test F1 @ selected queue": "{:.3f}",
    "Test AP": "{:.3f}",
    "Validation precision @ 5%": "{:.3f}",
    "Validation recall @ 5%": "{:.3f}",
    "Test precision @ 5%": "{:.3f}",
    "Test recall @ 5%": "{:.3f}",
    "Test accuracy @ 5%": "{:.3f}",
    "Test F1 @ 5%": "{:.3f}",
}))

fig, axes = plt.subplots(2, 2, figsize=(12, 10))
axes = axes.flatten()
for ax, (name, matrix) in zip(axes, confusion_matrices.items()):
    display_matrix = ConfusionMatrixDisplay(
        confusion_matrix=matrix,
        display_labels=["Not in incident window", "In incident window"],
    )
    display_matrix.plot(ax=ax, colorbar=False, values_format="d")
    ax.set_title(f"{name} - {selected_queue_rate:.1%} queue")
    ax.tick_params(axis="x", labelrotation=25)
plt.tight_layout()
plt.show()


# Notebook code cell 30
# Save the model comparison, including the validation-selected queue rate.
comparison_path = output_dir / "model_comparison.csv"
model_comparison.to_csv(comparison_path, index=False)

# Preserve test labels and scores, plus flags at both selected and 5% capacity.
comparison_best_model = model_comparison.iloc[0]["Model"]
selected_queue_rate = float(model_comparison.iloc[0]["Selected queue rate"])
best_risk_column = comparison_best_model.lower().replace(" ", "_") + "_risk"
scored_days = employee_daily.iloc[test_indices][
    ["user", "day", target_column]
].copy().reset_index(drop=True)
actual_labels = y_test.to_numpy()
selected_top_count = max(1, int(np.ceil(len(actual_labels) * selected_queue_rate)))
top_five_count = max(1, int(np.ceil(len(actual_labels) * 0.05)))

for name, scores in risk_scores.items():
    scores = np.asarray(scores)
    model_column = name.lower().replace(" ", "_")
    scored_days[f"{model_column}_risk"] = scores

    selected_indices = np.argsort(scores)[-selected_top_count:]
    selected_flags = np.zeros(len(scores), dtype=int)
    selected_flags[selected_indices] = 1
    scored_days[f"{model_column}_flagged_selected_queue"] = selected_flags

    five_percent_indices = np.argsort(scores)[-top_five_count:]
```

```
    five_percent_flags = np.zeros(len(scores), dtype=int)
    five_percent_flags[five_percent_indices] = 1
    scored_days[f"{model_column}_flagged_top_5_percent"] = five_percent_flags

scores_path = output_dir / "evaluation_employee_day_scores.csv"
scored_days.to_csv(scores_path, index=False)

# Save confusion counts at the selected queue capacity.
confusion_rows = []
for name, matrix in confusion_matrices.items():
    tn, fp, fn, tp = matrix.ravel()
    confusion_rows.append({
        "model": name,
        "review_queue_rate": selected_queue_rate,
        "true_negative": tn,
        "false_positive": fp,
        "false_negative": fn,
        "true_positive": tp,
    })
confusion_path = output_dir / "confusion_matrix_counts.csv"
pd.DataFrame(confusion_rows).to_csv(confusion_path, index=False)

risk_columns = [column for column in scored_days.columns if column.endswith("_risk")]
employee_ranking = (
    scored_days.groupby("user")
    .agg({**{column: "max" for column in risk_columns}, target_column: "max"})
    .sort_values(best_risk_column, ascending=False)
    .reset_index()
)
ranking_path = output_dir / "evaluation_employee_ranking.csv"
employee_ranking.to_csv(ranking_path, index=False)

print("Saved model comparison:", comparison_path)
print("Saved employee-day scores and queue flags:", scores_path)
print("Saved selected-capacity confusion matrix counts:", confusion_path)
print("Saved employee ranking:", ranking_path)
print("Model selected by validation AP:", comparison_best_model)
print(f"Queue rate selected by validation F1: {selected_queue_rate:.1%}")


# Notebook code cell 31
baseline_rate = float(y_test.mean())

print(f"Positive-label baseline: {baseline_rate:.2%}")

if baseline_rate == 0:
    print("Lift cannot be calculated because the evaluation set has no positive labels.")
else:
    model_comparison["Test AP lift over baseline"] = (
        model_comparison["Test AP"] / baseline_rate
    )

    for _, row in model_comparison.iterrows():
        print(
            f"{row['Model']}: "
            f"{row['Test AP lift over baseline']:.1f}x baseline"
        )

    display(
```

```
        model_comparison.style.format({
            "Validation AP": "{:.3f}",
            "Validation precision @ 5%": "{:.3f}",
            "Validation recall @ 5%": "{:.3f}",
            "Test precision @ 5%": "{:.3f}",
            "Test recall @ 5%": "{:.3f}",
            "Test accuracy": "{:.3f}",
            "Test F1": "{:.3f}",
            "Test AP": "{:.3f}",
            "Test AP lift over baseline": "{:.1f}x",
        })
    )

# Notebook code cell 33
import joblib

print("Retraining models on all labeled employee-day data...")

for name, model in models.items():
    if name == "Isolation Forest":
        model.fit(X)
    elif name == "Random Forest" and selected_synthetic_augmentation[name]:
        synthetic_X = make_synthetic_positive_counts(X, y, random_state=2027)
        augmented_X = pd.concat([X.reset_index(drop=True), synthetic_X], ignore_index=True)
        augmented_y = pd.concat(
            [y.reset_index(drop=True), pd.Series(np.ones(len(synthetic_X), dtype=int))],
            ignore_index=True,
        )
        model.fit(augmented_X, augmented_y)
        print(f"Random Forest final fit includes {len(synthetic_X):,} synthetic training rows.")
    else:
        model.fit(X, y)

model_bundle = {
    "models": models,
    "feature_columns": feature_columns,
    "model_names": list(models.keys()),
    "synthetic_augmentation": {
        "method": "Poisson/binomial count perturbation of positive training rows",
        "selected_by_validation": selected_synthetic_augmentation,
    },
}

model_path = output_dir / "insider_investigation_model.joblib"
joblib.dump(model_bundle, model_path)

print("Saved all four models to:", model_path)
print("Models saved:", ", ".join(model_bundle["model_names"]))


# Notebook code cell 34
from sklearn.metrics import classification_report

print(f"\n{name}")
print(classification_report(
    actual_labels,
    predicted_labels,
    labels=[0, 1],
    target_names=["Not in incident window", "In incident window"],
```

```
    zero_division=0,
    digits=3,
))
```

Listing 10.1: Notebook code cells for data preparation, model evaluation, and bundle creation

10.11 Scoring core source listing

This listing captures the Python functions used by the API and Streamlit interface to validate the model bundle, read activity data, create employee-day features, calculate ranks, and format activity details.

```
1
2
3  from pathlib import Path
4  import subprocess
5
6  import joblib
7  import numpy as np
8  import pandas as pd
9
10
11 REQUIRED_FILES = ["logon.csv", "device.csv", "file.csv", "email.csv", "http.csv"]
12 MODEL_NAMES = [
13     "Isolation Forest",
14     "Logistic Regression",
15     "Decision Tree",
16     "Random Forest",
17 ]
18 CHUNK_SIZE = 25_000
19
20
21 def choose_data_folder():
22     """Show a macOS folder picker. Data stays in the selected local folder."""
23     script = (
24         'POSIX path of (choose folder with prompt '
25         '"Choose the folder containing the five activity CSV files")'
26     )
27     result = subprocess.run(
28         ["osascript", "-e", script],
29         capture_output=True,
30         text=True,
31         check=False,
32     )
33     if result.returncode != 0:
34         return None
35     selected = result.stdout.strip()
36     return Path(selected) if selected else None
37
38
39 def load_model_bundle(model_path):
40     """Load and check the bundle created by notebook Cell 31."""
41     bundle = joblib.load(model_path)
42     if not isinstance(bundle, dict) or "models" not in bundle:
43         raise ValueError(
44             "This model file does not contain all four models. "
45             "Run notebook Cell 31, then restart the dashboard."
46         )
47     missing_models = [name for name in MODEL_NAMES if name not in bundle["models"]]
48     if missing_models:
49         raise ValueError("Model file is missing: " + ", ".join(missing_models))
50     if not bundle.get("feature_columns"):
51         raise ValueError("The model file is missing its feature column list.")
52     return bundle
53
54
55 def process_activity_file(file_path):
56     """Aggregate one source log into the same employee-day features as the notebook."""
57     file_path = Path(file_path)
58     source = file_path.stem.lower()
```

```
    headers = [str(c).strip().lower() for c in pd.read_csv(file_path, nrows=0).columns]
    useful = ["date", "user", "activity", "attachments"]
    selected = [column for column in useful if column in headers]

    if "date" not in selected or "user" not in selected:
        raise ValueError(f"{file_path.name} must contain 'date' and 'user' columns.")

    daily = None
    for chunk in pd.read_csv(
        file_path,
        usecols=lambda c: str(c).strip().lower() in selected,
        chunksize=CHUNK_SIZE,
        low_memory=False,
    ):
        chunk.columns = [str(c).strip().lower() for c in chunk.columns]
        chunk["user"] = chunk["user"].astype("string").str.strip()
        dates = pd.to_datetime(chunk["date"], errors="coerce", format="mixed")
        missing_user = chunk["user"].isna() | chunk["user"].eq("")
        keep = ~(missing_user | dates.isna() | chunk.duplicated())
        chunk = chunk.loc[keep].copy()
        dates = dates.loc[keep]
        if chunk.empty:
            continue

        chunk["day"] = dates.dt.strftime("%Y-%m-%d").to_numpy()
        hour = dates.dt.hour
        chunk["after_hours_count"] = ((hour < 6) | (hour >= 18)).astype("int8").to_numpy()
        chunk["event_count"] = 1
        columns = ["event_count", "after_hours_count"]

        if source == "device" and "activity" in chunk:
            chunk["device_connect_count"] = (
                chunk["activity"].astype("string").str.lower().eq("connect").astype("int8")
            )
            columns.append("device_connect_count")

        if source == "email" and "attachments" in chunk:
            chunk["attachment_count"] = (
                pd.to_numeric(chunk["attachments"], errors="coerce").fillna(0)
            )
            columns.append("attachment_count")

        part = chunk.groupby(["user", "day"])[columns].sum()
        part = part.rename(columns={c: f"{source}_{c}" for c in columns})
        daily = part if daily is None else daily.add(part, fill_value=0)

    if daily is None:
        return pd.DataFrame()
    return daily


def _load_optional_names(data_folder):
    """Read names only when a small, separate employee lookup file is present."""
    candidates = ["employees.csv", "employee.csv", "users.csv", "user.csv"]
    for filename in candidates:
        path = Path(data_folder) / filename
        if not path.is_file():
            continue
        lookup = pd.read_csv(path, low_memory=False)
```

```
        normalized = {str(c).strip().lower(): c for c in lookup.columns}
        user_col = next(
            (normalized[c] for c in ("user", "employee", "employee_id", "user_id") if c in normalized),
            None,
        )
        name_col = next(
            (normalized[c] for c in ("name", "employee_name", "full_name") if c in normalized),
            None,
        )
        if user_col and name_col:
            result = lookup[[user_col, name_col]].copy()
            result.columns = ["user", "name"]
            result["user"] = result["user"].astype("string").str.strip()
            result["name"] = result["name"].astype("string").str.strip()
            return result.dropna(subset=["user"]).drop_duplicates("user")
    return None


def _score(model, model_name, values):
    """Return larger values for higher risk or more unusual activity."""
    if model_name == "Isolation Forest":
        return -np.asarray(model.decision_function(values), dtype=float)
    if hasattr(model, "predict_proba"):
        return np.asarray(model.predict_proba(values)[:, 1], dtype=float)
    return np.asarray(model.decision_function(values), dtype=float)


def score_folder(data_folder, bundle, progress_callback=None):
    """Build employee-day features, score with all four models, and rank employees."""
    def report_progress(percent, message):
        if progress_callback is not None:
            progress_callback(percent, message)

    data_folder = Path(data_folder)
    missing = [name for name in REQUIRED_FILES if not (data_folder / name).is_file()]
    if missing:
        raise FileNotFoundError("Selected folder is missing: " + ", ".join(missing))

    sources = []
    for index, name in enumerate(REQUIRED_FILES):
        report_progress(index * 12, f"Reading and aggregating {name} ({index + 1} of {len(REQUIRED_FILES)})
")
        sources.append(process_activity_file(data_folder / name))
        report_progress((index + 1) * 12, f"Finished {name} ({index + 1} of {len(REQUIRED_FILES)})")
    sources = [frame for frame in sources if not frame.empty]
    if not sources:
        raise ValueError("The selected activity files contained no usable records.")

    employee_days = pd.concat(sources, axis=1).fillna(0)
    employee_days = employee_days.groupby(level=["user", "day"]).sum().reset_index()
    report_progress(65, "Preparing employee-day features")

    feature_columns = bundle["feature_columns"]
    for column in feature_columns:
        if column not in employee_days.columns:
            employee_days[column] = 0
    model_input = employee_days[feature_columns].apply(pd.to_numeric, errors="coerce").fillna(0)

    score_columns = []
```

```
    for index, name in enumerate(MODEL_NAMES):
        scores = _score(bundle["models"][name], name, model_input)
        # Percentiles put the four different score scales on a comparable rank scale.
        column = name.lower().replace(" ", "_") + "_rank"
        employee_days[column] = pd.Series(scores).rank(pct=True, method="average").to_numpy()
        score_columns.append(column)
        report_progress(65 + (index + 1) * 6, f"Scoring with {name} ({index + 1} of {len(MODEL_NAMES)})")

    employee_days["consensus_score"] = employee_days[score_columns].mean(axis=1)

    # Rank each employee by their highest-risk day for each model.
    aggregations = {
        "consensus_score": ("consensus_score", "max"),
        "active_days": ("day", "nunique"),
    }
    aggregations.update({column: (column, "max") for column in score_columns})
    ranking = employee_days.groupby("user").agg(**aggregations).reset_index()

    # This is a review-priority rule, not a probability or proof of wrongdoing.
    for column in score_columns:
        ranking[column + "_top_10"] = ranking[column].rank(pct=True, method="average") >= 0.90
    agreement_columns = [column + "_top_10" for column in score_columns]
    ranking["model_agreement"] = ranking[agreement_columns].sum(axis=1).astype(int)
    queue_size = max(1, int(np.ceil(len(ranking) * 0.05)))
    consensus_cutoff = ranking["consensus_score"].nlargest(queue_size).min()
    ranking["status"] = np.where(
        (ranking["consensus_score"] >= consensus_cutoff)
        | (ranking["model_agreement"] >= 2),
        "SUSPICIOUS",
        "REVIEW",
    )

    # Summarize activity signals on each employee's highest-consensus day.
    signals = [
        ("After-hours activity", "logon_after_hours_count"),
        ("USB connections", "device_device_connect_count"),
        ("File activity", "file_event_count"),
        ("Email activity", "email_event_count"),
        ("Web activity", "http_event_count"),
    ]
    reasons = {}
    for user, group in employee_days.groupby("user", sort=False):
        row = group.loc[group["consensus_score"].idxmax()]
        unusual = []
        for label, column in signals:
            if column not in employee_days.columns:
                continue
            value = float(row[column])
            cutoff = float(employee_days[column].quantile(0.95))
            if value > 0 and cutoff > 0 and value >= cutoff:
                unusual.append(f"{label} ({value:g} events on {row['day']})")
        reasons[user] = "; ".join(unusual[:3]) or (
            f"Activity ranked highly across the four models on {row['day']}"
        )

    ranking["why_flagged"] = ranking["user"].map(reasons)
    report_progress(92, "Building investigation report")
    names = _load_optional_names(data_folder)
    if names is not None:
```

```
        ranking = ranking.merge(names, on="user", how="left")
        ranking["name"] = ranking["name"].replace("", pd.NA)
    else:
        ranking["name"] = pd.NA

    ranking["_status_order"] = ranking["status"].map({"SUSPICIOUS": 0, "REVIEW": 1})
    ranking = ranking.sort_values(
        ["_status_order", "consensus_score"], ascending=[True, False]
    ).drop(columns="_status_order").reset_index(drop=True)

    employee_days = employee_days.merge(
        ranking[["user", "name", "status"]], on="user", how="left"
    )
    report_progress(100, "Analysis complete")
    return employee_days, ranking


def friendly_activity_table(user_days):
    labels = {
        "day": "Date",
        "consensus_score": "Combined rank",
        "logon_event_count": "Logins",
        "logon_after_hours_count": "After-hours",
        "device_device_connect_count": "USB connects",
        "file_event_count": "Files",
        "email_event_count": "Emails",
        "http_event_count": "Web visits",
    }
    columns = [column for column in labels if column in user_days.columns]
    return (
        user_days[columns]
        .sort_values("consensus_score", ascending=False)
        .head(20)
        .rename(columns=labels)
    )
```

Listing 10.2: Forensic core implementation

10.12 FastAPI source listing

This listing documents the local HTTP interface, file staging, progress state, API errors, and employee detail response.

```
"""Local API for the React interface; run with `uvicorn api:app --reload`."""

from pathlib import Path
from shutil import copyfileobj
from tempfile import TemporaryDirectory
from threading import Lock

import pandas as pd
from fastapi import FastAPI, File, HTTPException, UploadFile
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel

from forensic_core import MODEL_NAMES, REQUIRED_FILES, friendly_activity_table, load_model_bundle,
    score_folder


PROJECT_DIR = Path(__file__).resolve().parent
MODEL_PATH = PROJECT_DIR / "sprint1_output" / "insider_investigation_model.joblib"
RESULTS_DIR = PROJECT_DIR / "sprint1_output"

app = FastAPI(title="Forensio API", docs_url=None, redoc_url=None)
app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:5173", "http://127.0.0.1:5173"],
    allow_methods=["GET", "POST"],
    allow_headers=["Content-Type"],
)

analysis_lock = Lock()
employee_days: pd.DataFrame | None = None
ranking: pd.DataFrame | None = None
analysis_progress = {"percent": 0, "stage": "Waiting to start", "active": False}


class AnalyzeRequest(BaseModel):
    folder: str


class EmployeeRequest(BaseModel):
    user: str


def _records(frame: pd.DataFrame) -> list[dict]:
    clean = frame.astype(object).where(pd.notna(frame), None)
    return clean.to_dict(orient="records")


def _analyze_folder(folder: Path) -> dict:
    global employee_days, ranking, analysis_progress
    if not MODEL_PATH.is_file():
        raise HTTPException(status_code=503, detail="The four-model bundle is missing. Run notebook Cell 31
     first.")
    try:
        with analysis_lock:
            analysis_progress = {"percent": 15, "stage": "Preparing analysis", "active": True}
            bundle = load_model_bundle(MODEL_PATH)
            def update_progress(percent, stage):
                global analysis_progress
```

```
                analysis_progress = {
                    "percent": 15 + round(percent * 0.85),
                    "stage": stage,
                    "active": True,
                }

            days, report = score_folder(folder, bundle, progress_callback=update_progress)
            RESULTS_DIR.mkdir(exist_ok=True)
            days.to_csv(RESULTS_DIR / "dashboard_day_scores.csv", index=False)
            report.to_csv(RESULTS_DIR / "dashboard_report.csv", index=False)
            employee_days, ranking = days, report
            analysis_progress = {"percent": 100, "stage": "Analysis complete", "active": False}
    except HTTPException:
        raise
    except Exception as error:
        analysis_progress = {"percent": 0, "stage": "Analysis failed", "active": False}
        raise HTTPException(status_code=400, detail=f"Analysis could not be completed: {error}") from error

    suspicious = report[report["status"] == "SUSPICIOUS"]
    return {
        "employees": _records(report),
        "summary": {
            "employees": int(len(report)),
            "flagged": int(len(suspicious)),
            "review": int(len(report) - len(suspicious)),
            "activeDays": int(days["day"].nunique()),
        },
        "models": MODEL_NAMES,
    }


@app.get("/api/health")
def health():
    return {
        "ready": MODEL_PATH.is_file(),
        "message": "Model bundle is ready." if MODEL_PATH.is_file() else "Model bundle is missing. Run
     notebook Cell 31.",
    }


@app.get("/api/progress")
def progress():
    return analysis_progress


@app.post("/api/analyze")
def analyze(request: AnalyzeRequest):
    folder = Path(request.folder).expanduser()
    if not folder.is_dir():
        raise HTTPException(status_code=400, detail="That folder could not be found on this computer.")
    return _analyze_folder(folder)


@app.post("/api/analyze-upload")
def analyze_upload(files: list[UploadFile] = File(...)):
    expected = set(REQUIRED_FILES)
    selected: dict[str, UploadFile] = {}
    optional = {"employees.csv", "employee.csv", "users.csv", "user.csv"}
    for upload in files:
```

```
        name = Path(upload.filename or "").name.lower()
        if name in expected or name in optional:
            if name in selected:
                raise HTTPException(status_code=400, detail=f"The selected folder contains duplicate {name}
      files.")
            selected[name] = upload
    missing = sorted(expected - selected.keys())
    if missing:
        raise HTTPException(status_code=400, detail="The selected folder is missing: " + ", ".join(missing))


    try:
        with TemporaryDirectory(prefix="insider-risk-") as temporary:
            folder = Path(temporary)
            for name, upload in selected.items():
                with (folder / name).open("wb") as target:
                    copyfileobj(upload.file, target, length=1024 * 1024)
            return _analyze_folder(folder)
    finally:
        for upload in files:
            upload.file.close()


@app.post("/api/employee")
def employee_detail(request: EmployeeRequest):
    if ranking is None or employee_days is None:
        raise HTTPException(status_code=409, detail="Run an analysis before opening employee details.")
    selected = ranking[ranking["user"].astype(str) == request.user]
    if selected.empty:
        raise HTTPException(status_code=404, detail="Employee was not found in this report.")
    report_row = selected.iloc[0]
    days = employee_days[employee_days["user"].astype(str) == request.user]
    activity = friendly_activity_table(days)
    score_names = {
        "isolation_forest_rank": "Isolation Forest",
        "logistic_regression_rank": "Logistic Regression",
        "decision_tree_rank": "Decision Tree",
        "random_forest_rank": "Random Forest",
    }
    scores = [
        {"model": label, "score": round(float(report_row[column]) * 100, 1)}
        for column, label in score_names.items()
        if column in report_row
    ]
    name = report_row.get("name")
    return {
        "user": request.user,
        "name": None if pd.isna(name) else str(name),
        "status": str(report_row["status"]),
        "whyFlagged": str(report_row["why_flagged"]),
        "modelAgreement": int(report_row["model_agreement"]),
        "activeDays": int(report_row["active_days"]),
        "scores": scores,
        "activity": _records(activity),
    }
```

Listing 10.3: FastAPI service implementation

10.13 Streamlit source listing

The Streamlit interface is retained as an alternative local demonstration. Its output is also review-oriented and shows the model agreement and aggregate activity for selected users.

```
# LaTeX-safe listing derived from app.py; Unicode string literals use equivalent Python escapes.
from pathlib import Path

import pandas as pd
import streamlit as st

from forensic_core import (
    choose_data_folder,
    friendly_activity_table,
    load_model_bundle,
    score_folder,
)

PROJECT_DIR = Path(__file__).resolve().parent
MODEL_PATH = PROJECT_DIR / "sprint1_output" / "insider_investigation_model.joblib"
RESULTS_DIR = PROJECT_DIR / "sprint1_output"

st.set_page_config(page_title="Forensio", page_icon="\U0001F50E", layout="wide")
st.markdown("""
<style>
.stApp { background: #f3f6fa; color: #202a35; }
[data-testid="stSidebar"] { background: #10243b; }
[data-testid="stSidebar"] * { color: #f0f4f8 !important; }
.block-container { max-width: 1380px; padding-top: 1.8rem; }
h1, h2, h3 { color: #172330; letter-spacing: -.02em; }
[data-testid="stMetric"] { background: #ffffff; border: 1px solid #e2e9f0;
  border-radius: 14px; padding: 1rem; box-shadow: 0 3px 12px #1720330c; }
[data-testid="stMetricValue"] { color: #315f91; }
.stButton > button[kind="primary"] { background: #396a9e; border: 0;
  border-radius: 9px; font-weight: 650; }
[data-testid="stDataFrame"] { border: 1px solid #e2e9f0; border-radius: 10px; }
.report-card { background: #ffffff; border: 1px solid #e2e9f0;
  border-left: 5px solid #4779ad; border-radius: 12px; padding: 1rem 1.2rem;
  margin: .65rem 0; }
.report-status { color: #315f91; font-weight: 750; letter-spacing: .04em; }
.analysis-animation { display: flex; align-items: center; gap: .7rem; color: #315f91;
  font-weight: 650; margin: .35rem 0 .75rem; }
.analysis-pulse { display: inline-flex; gap: 4px; }
.analysis-pulse span { width: 7px; height: 7px; border-radius: 50%; background: #4779ad;
  animation: analysis-bounce 1s infinite ease-in-out; }
.analysis-pulse span:nth-child(2) { animation-delay: .15s; }
.analysis-pulse span:nth-child(3) { animation-delay: .3s; }
.analysis-track { height: 4px; overflow: hidden; border-radius: 4px; background: #dce7f1;
  margin: .45rem 0 .85rem; }
.analysis-track::after { content: ""; display: block; width: 38%; height: 100%;
  background: #4779ad; border-radius: inherit; animation: analysis-sweep 1.25s infinite ease-in-out; }
@keyframes analysis-bounce { 0%, 60%, 100% { transform: translateY(0); opacity: .55; }
  30% { transform: translateY(-4px); opacity: 1; } }
@keyframes analysis-sweep { from { transform: translateX(-110%); }
  to { transform: translateX(290%); } }
</style>
""", unsafe_allow_html=True)

st.title("Forensio")
st.caption("Four-model results prioritize investigation. A flag is not proof of wrongdoing.")
st.sidebar.markdown("### \U0001F50E Forensio")
st.sidebar.caption("Decision Tree \u00B7 Random Forest \u00B7 Logistic Regression \u00B7 Isolation Forest")

```

```
if not MODEL_PATH.is_file():
    st.error("The four-model file is missing. Run notebook Cell 31, then restart this app.")
    st.stop()

try:
    bundle = load_model_bundle(MODEL_PATH)
except Exception as error:
    st.error(str(error))
    st.stop()

if "analysis_done" not in st.session_state:
    st.session_state.analysis_done = False

st.subheader("Choose activity folder")
st.write("Choose the folder containing logon.csv, device.csv, file.csv, email.csv, and http.csv.")

if st.button("Choose folder", type="primary"):
    folder = choose_data_folder()
    if folder:
        st.session_state.data_folder = str(folder)
    else:
        st.info("No folder selected.")

folder_text = st.session_state.get("data_folder")
if folder_text:
    folder = Path(folder_text)
    st.success(f"Selected folder: {folder.name}")
    if st.button("Analyze folder", type="primary"):
        try:
            animation_slot = st.empty()
            animation_slot.markdown(
                '<div class="analysis-animation">Analyzing activity data'
                '<span class="analysis-pulse"><span></span><span></span><span></span></span></div>'
                '<div class="analysis-track"></div>',
                unsafe_allow_html=True,
            )
            progress_bar = st.progress(0, text="Starting analysis...")
            with st.spinner("Analyzing activity data..."):
                days, report = score_folder(
                    folder,
                    bundle,
                    progress_callback=lambda percent, message: progress_bar.progress(
                        percent, text=message
                    ),
                )
            st.session_state.employee_days = days
            st.session_state.ranking = report
            st.session_state.analysis_done = True
            RESULTS_DIR.mkdir(exist_ok=True)
            days.to_csv(RESULTS_DIR / "dashboard_day_scores.csv", index=False)
            report.to_csv(RESULTS_DIR / "dashboard_report.csv", index=False)
            progress_bar.progress(100, text="Analysis complete")
            animation_slot.empty()
            st.success("Analysis complete. The report files were saved in sprint1_output.")
        except Exception as error:
            if "animation_slot" in locals():
                animation_slot.empty()
            st.error(f"Analysis could not be completed: {error}")

```

```
if st.session_state.analysis_done:
    ranking = st.session_state.ranking
    employee_days = st.session_state.employee_days
    flagged = ranking[ranking["status"] == "SUSPICIOUS"].copy()

    st.divider()
    st.header("Investigation Report")
    st.caption(
        "The combined score is a relative rank, not a probability. SUSPICIOUS "
        "means prioritized for human review based on the four-model review rule."
    )

    top_ten = flagged.head(10)
    if top_ten.empty:
        st.info("No employee crossed the review rule for this dataset.")
    else:
        st.markdown("### Top 10 employees flagged for review")
        for _, row in top_ten.iterrows():
            employee_name = row.get("name")
            name_line = (
                f"<p><b>Name:</b> {employee_name}</p>"
                if pd.notna(employee_name) and str(employee_name).strip()
                else ""
            )
            st.markdown(
                f"""
                <div class="report-card">
                  <div class="report-status">Status: SUSPICIOUS</div>
                  <p><b>Employee:</b> {row['user']}</p>
                  {name_line}
                  <p><b>Flagged for:</b> {row['why_flagged']}</p>
                  <p><b>Model agreement:</b> {int(row['model_agreement'])} / 4</p>
                </div>
                """,
                unsafe_allow_html=True,
            )

    st.markdown("### Other employees")
    top_ten_ids = set(top_ten["user"].tolist())
    table = ranking[~ranking["user"].isin(top_ten_ids)].head(25).copy()
    table["Combined rank"] = (table["consensus_score"] * 100).round(1)
    table["Model agreement"] = table["model_agreement"].astype(str) + " / 4"
    table = table.rename(columns={
        "user": "Employee",
        "name": "Name",
        "status": "Status",
        "why_flagged": "Flagged for",
        "active_days": "Active days",
    })
    shown = [
        column for column in
        ["Status", "Employee", "Name", "Combined rank", "Model agreement", "Active days", "Flagged for"]
        if column in table.columns
    ]
    if table.empty:
        st.caption("There are no additional employees to list.")
    else:
        st.dataframe(table[shown], hide_index=True, use_container_width=True)

```

```
    selected = st.selectbox("Review employee activity", ranking["user"].tolist())
    selected_days = employee_days[employee_days["user"] == selected].copy()
    selected_report = ranking.loc[ranking["user"] == selected].iloc[0]
    st.subheader(f"Activity details: {selected}")
    st.write(f"**Status:** {selected_report['status']}")
    st.write(f"**Flagged for:** {selected_report['why_flagged']}")
    if pd.notna(selected_report["name"]) and str(selected_report["name"]).strip():
        st.write(f"**Name:** {selected_report['name']}")

    model_columns = [
        "isolation_forest_rank",
        "logistic_regression_rank",
        "decision_tree_rank",
        "random_forest_rank",
    ]
    model_scores = selected_report[model_columns].rename({
        "isolation_forest_rank": "Isolation Forest",
        "logistic_regression_rank": "Logistic Regression",
        "decision_tree_rank": "Decision Tree",
        "random_forest_rank": "Random Forest",
    }).to_frame("Rank score")
    model_scores["Rank score"] = (model_scores["Rank score"] * 100).round(1)
    st.markdown("**Scores from each model**")
    st.dataframe(model_scores, use_container_width=True)

    st.markdown("**Highest-ranked activity days**")
    st.dataframe(
        friendly_activity_table(selected_days),
        hide_index=True,
        use_container_width=True,
    )

st.caption(
    "Model scores are relative ranks within the selected data. Confirm findings "
    "against original logs and other evidence before drawing conclusions."
)
```

Listing 10.4: Alternative Streamlit interface

10.14 React application source listing

The React client implements service health checks, directory upload, progress polling, report rendering, employee selection, search, and filtering. The listing is included to show how the browser workflow relates to the API contract described in Chapter 4.

```
import { useEffect, useMemo, useRef, useState } from 'react'

const api = async (url, options) => {
  const isForm = options?.body instanceof FormData
  let response
  try {
    response = await fetch(url, {
      ...options,
      headers: { ...(isForm ? {} : { 'Content-Type': 'application/json' }), ...options?.headers },
    })
  } catch {
    throw new Error('Cannot reach the local API at 127.0.0.1:8000. Start the app with "npm run dev" from the frontend folder.')
  }
  let data
  try {
    data = await response.json()
  } catch {
    throw new Error('The local API returned an unreadable response. Check the API terminal for details.')
  }
  if (!response.ok) throw new Error(data.detail || 'Something went wrong.')
  return data
}

const formatUser = (value) => value?.split('\\').at(-1) || value

function App() {
  const [health, setHealth] = useState(null)
  const [selectedFiles, setSelectedFiles] = useState([])
  const [folderName, setFolderName] = useState('')
  const [data, setData] = useState(null)
  const [selected, setSelected] = useState(null)
  const [detail, setDetail] = useState(null)
  const [query, setQuery] = useState('')
  const [filter, setFilter] = useState('All employees')
  const [busy, setBusy] = useState(false)
  const [analysisProgress, setAnalysisProgress] = useState({ percent: 0, stage: 'Preparing your files' })
  const [elapsedSeconds, setElapsedSeconds] = useState(0)
  const analysisStartedAt = useRef(null)
  const [error, setError] = useState('')

  useEffect(() => {
    let active = true
    let timer
    const check = async () => {
      try {
        const result = await api('/api/health')
        if (active) setHealth({ ...result, available: true })
      } catch {
        if (active) {
          setHealth({ ready: false, available: false })
          timer = window.setTimeout(check, 4000)
        }
      }
    }
    check()
    return () => { active = false; window.clearTimeout(timer) }
  }, [])
```

```
58
59  useEffect(() => {
60    if (!selected) return
61    setDetail(null)
62    api('/api/employee', { method: 'POST', body: JSON.stringify({ user: selected.user }) })
63      .then(setDetail).catch((err) => setError(err.message))
64  }, [selected])
65
66  useEffect(() => {
67    if (!busy) return
68    const timer = window.setInterval(() => {
69      if (analysisStartedAt.current) {
70        setElapsedSeconds(Math.floor((Date.now() - analysisStartedAt.current) / 1000))
71      }
72    }, 1000)
73    return () => window.clearInterval(timer)
74  }, [busy])
75
76  const employees = data?.employees || []
77  const requiredFiles = ['logon.csv', 'device.csv', 'file.csv', 'email.csv', 'http.csv']
78  const selectedNames = new Set(selectedFiles.map((file) => file.name.toLowerCase()))
79  const missingFiles = requiredFiles.filter((name) => !selectedNames.has(name))
80  const visibleEmployees = useMemo(() => employees.filter((person) => {
81    const matchesQuery = `${person.user} ${person.name || ''}`.toLowerCase().includes(query.toLowerCase())
82    const matchesFilter = filter === 'All employees' || person.status === filter.toUpperCase()
83    return matchesQuery && matchesFilter
84  }), [employees, filter, query])
85
86  async function analyze(event) {
87    event.preventDefault()
88    setBusy(true)
89    setError('')
90    setDetail(null)
91    setSelected(null)
92    analysisStartedAt.current = Date.now()
93    setElapsedSeconds(0)
94    setAnalysisProgress({ percent: 0, stage: 'Preparing your files' })
95    let polling = true
96    const pollProgress = async () => {
97      while (polling) {
98        try {
99          const progress = await api('/api/progress')
100         if (progress.active) {
101           setAnalysisProgress({ percent: Math.max(15, progress.percent), stage: progress.stage })
102         }
103       } catch { /* The analysis request will report API errors. */ }
104       await new Promise((resolve) => window.setTimeout(resolve, 400))
105     }
106   }
107   const progressPolling = pollProgress()
108   try {
109     const form = new FormData()
110     selectedFiles.forEach((file) => form.append('files', file, file.name))
111     const result = await new Promise((resolve, reject) => {
112       const request = new XMLHttpRequest()
113       request.open('POST', '/api/analyze-upload')
114       request.upload.onprogress = (progress) => {
115         if (progress.lengthComputable) {
116           setAnalysisProgress({
```

```
              percent: Math.round((progress.loaded / progress.total) * 15),
              stage: 'Uploading selected logs',
            })
          }
        }
        request.onload = () => {
          let body
          try { body = JSON.parse(request.responseText) } catch {
            reject(new Error('The local API returned an unreadable response. Check the API terminal for details.'))
            return
          }
          if (request.status < 200 || request.status >= 300) {
            reject(new Error(body.detail || 'Something went wrong.'))
            return
          }
          setAnalysisProgress({ percent: 100, stage: 'Analysis complete' })
          resolve(body)
        }
        request.onerror = () => reject(new Error('Cannot reach the local API at 127.0.0.1:8000. Start the app with "npm run dev" from the frontend folder.'))
        request.send(form)
      })
      setData(result)
      setFilter('All employees')
    } catch (err) { setError(err.message) }
    finally {
      polling = false
      await progressPolling
      setBusy(false)
    }
  }

  const flagged = employees.filter((person) => person.status === 'SUSPICIOUS').slice(0, 10)
  const otherEmployees = visibleEmployees.filter((person) => !flagged.includes(person))
  const remainingSeconds = analysisProgress.percent >= 20 && elapsedSeconds >= 3
    ? Math.max(1, Math.ceil(elapsedSeconds * (100 - analysisProgress.percent) / analysisProgress.percent))
    : null
  const remainingLabel = remainingSeconds === null
    ? 'Estimating time remaining...'
    : remainingSeconds >= 60
      ? `About ${Math.ceil(remainingSeconds / 60)} min remaining`
      : `About ${remainingSeconds} sec remaining`

  return (
    <div className="app-shell">
      <aside className="sidebar">
        <div className="brand"><span className="brand-mark">( )</span><span>Forensio</span></div>
        <div className="workspace-label">WORKSPACE</div>
        <button className="nav-item active"><span className="nav-icon">[grid]</span>Overview</button>
        <div className="side-rule" />
        <div className="side-caption">ANALYSIS MODELS</div>
        <div className="model-nav"><i className="dot teal" />Isolation Forest</div>
        <div className="model-nav"><i className="dot blue" />Logistic Regression</div>
        <div className="model-nav"><i className="dot violet" />Decision Tree</div>
        <div className="model-nav"><i className="dot amber" />Random Forest</div>
        <div className="sidebar-bottom"><div className="privacy-icon"><></div><div><b>Private by design</b><small>Activity data stays on this device.</small></div></div>
      </aside>
```

```
 <main className="main-content">
   <header className="topbar"><div className="crumb">Risk intelligence <span>/</span> Overview</div><
div className="top-right"><span className="local-badge"><i /> Local analysis</span><div className="
avatar">HR</div></div></header>
   <div className="page-wrap">
     <section className="page-heading"><div><div className="eyebrow">INSIDER RISK . REVIEW CONSOLE</
div><h1>Investigation overview</h1><p>Bring unusual activity into focus. Every signal is a prompt for
careful human review.</p></div><div className="date-chip">   Model ensemble <span>.</span> 4
models</div></section>

     <form className="source-card" onSubmit={analyze}>
       <div className="source-icon">H</div><div className="source-copy"><b>Choose an activity data
folder</b><span>Select the folder containing logon.csv, device.csv, file.csv, email.csv and http.csv.</
span></div>
       <div className="source-controls"><label className="folder-picker"><span>[folder]   {
folderName || 'Choose folder'}</span><input type="file" aria-label="Choose activity folder"
webkitdirectory="" directory="" multiple onChange={(event) => { const files = Array.from(event.target.
files || []); setSelectedFiles(files); setFolderName(files[0]?.webkitRelativePath?.split('/')[0] || '')
 }} /></label><button className="primary-button" disabled={busy || missingFiles.length > 0 || !health?.
ready}>{busy ? <><span className="spinner" /> Analyzing...</> : <>Analyze folder <span>-></span></>}</
button></div>
       {folderName && <div className="source-note">{missingFiles.length ? `Selected "${folderName}" .
${selectedFiles.length} files . missing ${missingFiles.join(', ')}` : `Selected "${folderName}" . ${
selectedFiles.length} files . all required logs found. Files upload only to this local app.`}</div>}
       {!health?.ready && <div className="source-note warning">{health?.available ? health.message :
health ? error || 'Local API is not reachable. Start the app with "npm run dev" from the frontend
folder.' : 'Checking the local analysis service...'}</div>}
     </form>
     {busy && <section className="analysis-card" role="status" aria-live="polite">
       <div className="analysis-visual" aria-hidden="true">
         <div className="analysis-orbit orbit-a" /><div className="analysis-orbit orbit-b" />
         <div className="analysis-core"><span>( )</span></div>
         <i className="analysis-node node-a" /><i className="analysis-node node-b" />
         <i className="analysis-node node-c" /><i className="analysis-node node-d" />
       </div>
       <div className="analysis-copy">
         <div className="eyebrow">LIVE ANALYSIS . FOUR MODEL ENSEMBLE</div>
         <h2>Finding patterns in the activity</h2>
         <p>{analysisProgress.stage}. Your files are processed locally.</p>
         <div className="analysis-progress-heading"><span>Analysis progress</span><b>{analysisProgress.
percent}%</b></div>
         <div className="analysis-progress-track" role="progressbar" aria-label="Analysis progress"
aria-valuemin="0" aria-valuemax="100" aria-valuenow={analysisProgress.percent}>
           <i style={{ width: `${analysisProgress.percent}%` }} />
         </div>
         <div className="analysis-time-remaining"><span className="time-pulse" />{remainingLabel}<
small>Estimate</small></div>
         <div className="analysis-stage-list"><span className={analysisProgress.percent >= 15 ? '
complete' : 'current'}>01 <b>Prepare</b></span><span className={analysisProgress.percent >= 65 ? '
complete' : analysisProgress.percent >= 15 ? 'current' : ''}>02 <b>Score</b></span><span className={
analysisProgress.percent >= 90 ? 'complete' : analysisProgress.percent >= 65 ? 'current' : ''}>03 <b>
Prioritize</b></span></div>
       </div>
     </section>}
     <details className="model-guide">
       <summary><span><b>How the model ensemble works</b><small>See how activity records become a
review priority.</small></span><span className="guide-action">Explore the scoring flow <i>+</i></span
></summary>
```

```
206             <div className="guide-content">
207               <div className="flow-strip">
208                 <div className="flow-step"><small>01 . SOURCES</small><b>Five activity logs</b><span>Logon .
     device . file . email . web</span></div>
209                 <i className="flow-arrow">-></i>
210                 <div className="flow-step"><small>02 . PREPARE</small><b>Employee-day features</b><span>
    Counts, after-hours, USB, attachments</span></div>
211                 <i className="flow-arrow">-></i>
212                 <div className="flow-step"><small>03 . SCORE</small><b>Four model views</b><span>Same
    feature row, scored independently</span></div>
213                 <i className="flow-arrow">-></i>
214                 <div className="flow-step"><small>04 . PRIORITIZE</small><b>Percentile consensus</b><span>
    Ranked queue for human review</span></div>
215               </div>
216               <div className="guide-section-title"><b>What each model contributes</b><span>Different
    methods can surface different patterns.</span></div>
217               <div className="model-explain-grid">
218                 <article className="model-explain indigo"><span>01</span><div><small>UNSUPERVISED ANOMALY</
    small><h3>Isolation Forest</h3><p>Finds employee-days that are isolated from common activity patterns.
    It does not use incident labels when learning what looks unusual.</p></div></article>
219                 <article className="model-explain sky"><span>02</span><div><small>SUPERVISED CLASSIFIER</
    small><h3>Logistic Regression</h3><p>Learns how activity features relate to labeled incidents and
    produces a risk score from that weighted relationship.</p></div></article>
220                 <article className="model-explain violet"><span>03</span><div><small>SUPERVISED CLASSIFIER</
    small><h3>Decision Tree</h3><p>Uses a sequence of learned feature thresholds to score activity. Its
    single tree is capped at depth 10.</p></div></article>
221                 <article className="model-explain amber"><span>04</span><div><small>SUPERVISED ENSEMBLE</
    small><h3>Random Forest</h3><p>Averages predictions from 200 decision trees trained with variation,
    giving a broader view than one tree alone.</p></div></article>
222               </div>
223               <div className="guide-rule"><span>How a review flag is assigned</span><p>For each employee-
    day, the four model scores become percentile ranks and are averaged. The employee list uses each person
    's highest-risk day. An employee is prioritized if that consensus is in the top 5%, or at least two
    model ranks are in the top 10%. These are relative review thresholds, not probabilities or findings of
    misconduct.</p></div>
224             </div>
225           </details>
226           {error && <div className="error-banner"><span>!</span>{error}<button onClick={() => setError('')}
     aria-label="Dismiss">x</button></div>}
227
228           {data ? <>
229             <section className="metrics-grid">
230               <Metric label="EMPLOYEES ANALYZED" value={data.summary.employees.toLocaleString()} detail="
    Across all activity sources" icon="P" tone="teal" />
231               <Metric label="PRIORITIZED FOR REVIEW" value={data.summary.flagged.toLocaleString()} detail="
    Flagged by the review rule" icon="" tone="orange" />
232               <Metric label="ACTIVE DAYS" value={data.summary.activeDays.toLocaleString()} detail="Employee-
    day records covered" icon="[grid]" tone="blue" />
233               <Metric label="MODELS IN AGREEMENT" value="4" detail="Independent scoring perspectives" icon
    ="(o)" tone="violet" />
234             </section>
235
236             <div className="section-heading"><div><div className="eyebrow">REVIEW QUEUE</div><h2>Employees
    prioritized for review</h2><p>Ranked by consensus across four models. A flag is not proof of wrongdoing
    .</p></div><span className="queue-count">{flagged.length} prioritized</span></div>
237             <aside className="agreement-note"><span className="agreement-note-icon">( )</span><div><b>Why
    model agreement is included</b><p>Each model looks for patterns differently. Agreement counts how many
    models place an employee's highest-risk day in their own top 10%; two or more agreeing can qualify an
    employee for review. It helps surface signals shared across methods, but it is a prioritization aid-not
```

```
 a probability or proof of wrongdoing.</p></div></aside>
238          {flagged.length ? <div className="flagged-grid">{flagged.map((person, i) => <button className={‘
flag-card ${selected?.user === person.user ? ’chosen’ : ’’}‘} key={person.user} onClick={() =>
setSelected(person)}><span className="flag-top"><span className="rank-number">{String(i + 1).padStart
(2, ’0’)}</span><span className="status-pill suspicious"><i /> Prioritize</span></span><b className="
employee-name">{person.name || formatUser(person.user)}</b><span className="employee-id">{person.name ?
 formatUser(person.user) : ’Employee identifier’}</span><span className="reason">{person.why_flagged}</
span><span className="flag-foot"><span>{person.model_agreement}/4 models aligned</span><span className
="score">{(person.consensus_score * 100).toFixed(1)} <small>rank</small></span></span></button>)}</div>
 : <div className="empty-state">No employees crossed the review rule for this dataset.</div>}
239
240          <section className="directory-card"><div className="directory-heading"><div><h2>Employee
directory</h2><p>Explore the full set of ranked employees.</p></div><div className="directory-controls
"><label className="search-box"><span>?</span><input value={query} onChange={(e) => setQuery(e.target.
value)} placeholder="Search employees" /></label><select value={filter} onChange={(e) => setFilter(e.
target.value)} aria-label="Filter employee status"><option>All employees</option><option>Suspicious</
option><option>Review</option></select></div></div><div className="table-scroll"><table><thead><tr><th>
EMPLOYEE</th><th>STATUS</th><th>CONSENSUS RANK</th><th>MODEL AGREEMENT</th><th>ACTIVE DAYS</th><th></th
></tr></thead><tbody>{otherEmployees.slice(0, 50).map((person) => <tr key={person.user} onClick={() =>
setSelected(person)} className={selected?.user === person.user ? ’selected-row’ : ’’}><td><b>{person.
name || formatUser(person.user)}</b><small>{person.name ? formatUser(person.user) : ’Employee
identifier’}</small></td><td><span className={‘status-pill ${person.status.toLowerCase()}‘}><i />{
person.status === ’SUSPICIOUS’ ? ’Prioritize’ : ’Review’}</span></td><td><div className="table-score"><
div><i style={{ width: ‘${Math.max(4, person.consensus_score * 100)}%‘ }} /></div><span>{(person.
consensus_score * 100).toFixed(1)}</span></div></td><td>{person.model_agreement} / 4 models</td><td>{
person.active_days}</td><td className="row-arrow">
```

Listing 10.5: React dashboard application

10.15 Interface style source listing

The frontend styling is maintained separately from the React component. These listings record the visual layout and the current theme overrides used by the saved UI screenshots.

```
@import url('https://fonts.googleapis.com/css2?family=DM+Sans:wght@400;500;600;700&family=Manrope:wght@400;500;600;700;800&display=swap');

:root{font-family:'DM Sans',sans-serif;color:#1e293b;background:#f4f5fb;font-synthesis:none;text-rendering:optimizeLegibility;font-optical-sizing:auto;font-weight:400;--ink:#20243b;--muted:#7e8b9c;--line:#e8edf2;--teal:#5b5dd2;--teal-dark:#4849b8;--orange:#e08d40;--surface:#fff}*{box-sizing:border-box}body{margin:0;min-width:320px;min-height:100vh}button,input,select{font:inherit}button{cursor:pointer}.app-shell{min-height:100vh}.sidebar{position:fixed;inset:0 auto 0 0;width:244px;background:#1a1c35;color:#d9e4ee;padding:27px 17px;display:flex;flex-direction:column;z-index:5}.brand{display:flex;align-items:center;gap:10px;color:#fff;font:800 18px Manrope,sans-serif;padding:0 11px 54px;letter-spacing:-.5px}.brand-mark{height:31px;width:31px;background:#5b5dd2;border-radius:10px;display:grid;place-items:center;font:24px Georgia;color:#fff}.brand-light{font-weight:500;color:#a7b9c9;margin-left:3px}.workspace-label,.side-caption{color:#8497aa;font-size:9px;letter-spacing:1.5px;font-weight:700;padding:0 12px 13px}.nav-item{border:0;background:transparent;color:#c1d0de;width:100%;padding:12px 13px;text-align:left;border-radius:8px;display:flex;align-items:center;gap:12px;font-size:13px}.nav-item.active{background:#292c50;color:#fff;font-weight:600;box-shadow:inset 3px 0 #8b84f2}.nav-icon{font-size:17px}.side-rule{border-top:1px solid #2c3c4c;margin:25px 10px 25px}.side-caption{padding-bottom:15px}.model-nav{color:#b9c6d2;font-size:12px;padding:10px 14px;display:flex;align-items:center;gap:11px}.dot{width:7px;height:7px;border-radius:50%;display:inline-block}.dot.teal{background:#4cc2b0}.dot.blue{background:#71a8ed}.dot.violet{background:#ae93e8}.dot.amber{background:#e8b16e}.sidebar-bottom{margin-top:auto;border-top:1px solid #304151;padding:19px 8px 4px;display:flex;gap:10px;align-items:center}.privacy-icon{width:31px;height:31px;border-radius:9px;background:#292c50;color:#9691f1;display:grid;place-items:center;font-size:20px}.sidebar-bottom b,.sidebar-bottom small{display:block}.sidebar-bottom b{font-size:11px;color:#e1ebf2;font-weight:600}.sidebar-bottom small{font-size:10px;color:#93a5b5;margin-top:4px}.main-content{margin-left:244px;min-height:100vh}.topbar{height:66px;border-bottom:1px solid #e8edf1;background:#fff;display:flex;align-items:center;justify-content:space-between;padding:0 42px}.crumb{font-size:12px;color:#768497}.crumb span{padding:0 9px;color:#c1cad4}.top-right{display:flex;align-items:center;gap:17px}.local-badge{font-size:10px;color:#637286;background:#f4f7f8;padding:7px 11px;border-radius:30px}.local-badge i{display:inline-block;width:6px;height:6px;background:#18a58e;border-radius:50%;margin-right:6px;vertical-align:1px}.avatar{height:30px;width:30px;border-radius:50%;background:#e3f1ee;color:#267d72;font-size:10px;font-weight:700;display:grid;place-items:center}.page-wrap{max-width:1460px;margin:auto;padding:36px 42px 28px}.page-heading{display:flex;justify-content:space-between;align-items:flex-end;margin-bottom:24px}.eyebrow{font-size:9px;font-weight:700;letter-spacing:1.45px;color:#8391a0}.page-heading h1{font:700 27px Manrope,sans-serif;letter-spacing:-.9px;margin:8px 0 6px;color:#1a293b}.page-heading p,.section-heading p,.directory-heading p{margin:0;color:#8390a0;font-size:12px}.date-chip{font-size:10px;border:1px solid #e3e9ed;border-radius:20px;color:#647487;background:#fff;padding:9px 12px;margin-bottom:2px}.date-chip span{color:#c2cbd3;padding:0 3px}.source-card{display:grid;grid-template-columns:42px minmax(230px,1fr) minmax(360px,1.4fr);align-items:center;gap:14px;background:#fff;border:1px solid #e7ecf0;border-radius:11px;padding:18px 19px;margin-bottom:22px;box-shadow:0 2px 8px #233b5205}.source-icon{width:39px;height:39px;border-radius:10px;background:#eeefff;color:#555ac9;display:grid;place-items:center;font-size:21px}.source-copy b,.source-copy span{display:block}.source-copy b{font-size:12px;color:#263547;margin-bottom:5px}.source-copy span{font-size:10px;color:#8a97a6;line-height:1.5;max-width:440px}.source-controls{display:flex;gap:8px}.source-controls input{min-width:0;flex:1;border:1px solid #dfe6eb;background:#fcfdfd;border-radius:7px;padding:10px 12px;font-size:11px;color:#364457;outline:none}.source-controls input:focus,.search-box:focus-within{border-color:#7bc4ba;box-shadow:0 0 0 3px #0f9b8e12}.source-controls input::placeholder{color:#adb7c1}.primary-button{border:0;background:#595bd0;border-radius:7px;color:white;font-size:11px;font-weight:600;padding:0 15px;white-space:nowrap;box-shadow:0 2px 4px #5556c624}.primary-button:hover{background:#4849b8}.primary-button:disabled{opacity:.55;cursor:not-allowed}.primary-button span{font-size:15px;margin-left:7px}.source-note{grid-column:2 / -1;color:#a96823;font-size:10px}.metrics-grid{display:grid;grid-template-columns:repeat(4,minmax(0,1fr));gap:13px;margin-bottom:34px}.metric-card{background:#fff;border:1px solid #e7ecf0;border-radius:10px;padding:16px 18px 15px;min-height:117px;box-shadow:0 2px 8px #233b5204}.metric-top{display:flex;align-items:center;justify-content:space-between;color:#8895a4;font-size:8px;letter-spacing:.9px;font-weight:700}.metric-icon{height:27px;width:27px;border-radius:8px;display:grid;place-items:center;font-style:normal;font-size:14px}.metric-icon.teal{background:#eeefff;color:#555ac9}.metric-icon.orange{background:#fff3e8;color:#d88639}.metric-icon.blue{background:#ecf3fc;color:#5d8dc9}.metric-icon.violet{background:#f2edfb;color:#9375c9}.metric-card strong{display:block;font:700 24px Manrope,sans-serif;color:#243346;margin:8px 0 3px;letter-spacing:-.6
```

```
px}.metric-card small{font-size:9px;color:#929eab}.section-heading{display:flex;justify-content:space-
between;align-items:flex-end;margin:0 1px 15px}.section-heading h2,.directory-heading h2,.detail-
heading h2,.welcome-card h2{font:700 17px Manrope,sans-serif;letter-spacing:-.4px;color:#243346;margin
:6px 0 5px}.section-heading p{font-size:11px}.queue-count{font-size:10px;background:#fff5eb;color:#
b66c29;border:1px solid #f5e1ce;padding:6px 10px;border-radius:18px;margin-bottom:3px}.flagged-grid{
display:grid;grid-template-columns:repeat(5,minmax(0,1fr));gap:11px;margin-bottom:29px}.flag-card{
background:#fff;border:1px solid #e7ecf0;border-radius:9px;padding:14px 13px 11px;text-align:left;min-
width:0;transition:border .15s,transform .15s,box-shadow .15s;box-shadow:0 1px 4px #20324805}.flag-card
:hover,.flag-card.chosen{border-color:#e0a76f;transform:translateY(-1px);box-shadow:0 4px 12px #45361
c0a}.flag-top{display:flex;align-items:center;justify-content:space-between;margin-bottom:13px}.rank-
number{font:700 10px Manrope;color:#b1bbc5}.status-pill{display:inline-flex;align-items:center;gap:5px;
border-radius:20px;padding:4px 7px;font-size:8px;font-weight:600;white-space:nowrap}.status-pill i{
width:5px;height:5px;border-radius:50%;background:currentColor}.status-pill.suspicious{background:#
fff3e8;color:#b96e30}.status-pill.review{background:#eef3f7;color:#64778b}.employee-name{display:block;
overflow:hidden;text-overflow:ellipsis;white-space:nowrap;color:#26364a;font-size:11px}.employee-id{
display:block;font-size:8px;color:#9ba6b0;margin-top:4px;overflow:hidden;text-overflow:ellipsis;white-
space:nowrap}.reason{display:-webkit-box;-webkit-line-clamp:2;-webkit-box-orient:vertical;overflow:
hidden;min-height:28px;color:#758394;font-size:9px;line-height:1.5;margin:12px 0 11px}.flag-foot{border
-top:1px solid #f0f2f4;padding-top:9px;display:flex;justify-content:space-between;align-items:center;
color:#939eaa;font-size:8px}.score{font:700 11px Manrope;color:#42586d}.score small{font:400 8px 'DM
Sans',sans-serif;color:#9aa5af}.empty-state,.table-empty{padding:27px;text-align:center;background:#fff
;color:#8390a0;font-size:12px;border:1px solid var(--line);border-radius:9px;margin-bottom:25px}.
directory-card,.detail-card{background:#fff;border:1px solid #e7ecf0;border-radius:10px;box-shadow:0 2
px 8px #233b5204;margin-bottom:20px}.directory-heading{display:flex;align-items:center;justify-content:
space-between;padding:18px 19px 15px}.directory-heading h2{font-size:14px;margin:0 0 4px}.directory-
heading p{font-size:10px}.directory-controls{display:flex;gap:8px}.search-box{height:32px;border:1px
solid #e2e8ed;border-radius:6px;display:flex;align-items:center;padding:0 8px;gap:6px;color:#91a0ae}.
search-box span{font-size:17px}.search-box input{border:0;outline:0;width:135px;font-size:10px;color:#3
b4a5d}.search-box input::placeholder{color:#a6b0ba}.directory-controls select{height:32px;border:1px
solid #e2e8ed;border-radius:6px;background:#fff;color:#6e7d8f;font-size:10px;padding:0 9px;outline:none
}.table-scroll{overflow:auto}table{border-collapse:collapse;width:100%;text-align:left}thead{background
:#f8fafb}th{font-size:8px;letter-spacing:.8px;color:#94a0ad;font-weight:700;padding:10px 14px;white-
space:nowrap;border-top:1px solid #eff2f4;border-bottom:1px solid #eff2f4}td{padding:10px 14px;border-
bottom:1px solid #f0f2f4;color:#68788b;font-size:10px;white-space:nowrap}tbody tr{cursor:pointer}tbody
tr:hover,tbody tr.selected-row{background:#f7fbfa}td:first-child{min-width:160px}td:first-child b,td:
first-child small{display:block}td:first-child b{font-size:10px;color:#364659;font-weight:600}td:first-
child small{font-size:8px;color:#9da8b3;margin-top:3px}.table-score{display:flex;align-items:center;gap
:8px}.table-score>div{width:74px;height:4px;background:#edf1f3;border-radius:4px}.table-score i{display
:block;height:100%;background:#6668d9;border-radius:4px}.table-score span{font-size:9px;color:#738296}.
row-arrow{font-size:13px;color:#9cabb7}.table-foot{padding:11px 14px;color:#96a1ad;font-size:9px}.table
-empty{margin:10px;border:0}.detail-card{padding:20px}.detail-heading{display:flex;justify-content:
space-between;align-items:flex-start}.detail-heading h2{margin:5px 0;font-size:17px}.detail-heading p{
font-size:10px;color:#8996a5;margin:0}.close-button{border:0;background:#f3f6f7;width:27px;height:27px;
border-radius:50%;font-size:18px;color:#768697}.detail-reason{display:flex;align-items:center;gap:11px;
padding:12px;background:#fbf8f4;border:1px solid #f2e9df;border-radius:7px;margin:17px 0;color:#6f7c8b;
font-size:10px}.detail-columns{display:grid;grid-template-columns:.85fr 1.15fr;gap:30px}.detail-columns
 h3{font-size:11px;color:#405166;margin:4px 0 15px}.detail-columns h3 small{font-size:8px;font-weight
:400;color:#9aa5af;margin-left:8px}.score-list{display:grid;gap:13px}.score-row{display:grid;grid-
template-columns:130px 1fr 35px;align-items:center;gap:10px;font-size:9px;color:#738296}.score-row b{
text-align:right;font-size:9px;color:#405166}.bar-track{height:6px;border-radius:8px;background:#edf1f3
;overflow:hidden}.bar-track i{height:100%;display:block;border-radius:8px;background:linear-gradient(90
deg,#8d86ed,#585bd2)}.activity-table{max-height:250px;overflow:auto;border:1px solid #eef1f3;border-
radius:6px}.activity-table th{font-size:7px;padding:8px}.activity-table td{font-size:8px;padding:7px}.
loading-detail{padding:28px;color:#8996a5;font-size:10px}.welcome-card{min-height:280px;background:#fff
;border:1px solid #e7ecf0;border-radius:11px;display:grid;grid-template-columns:230px 1fr;align-items:
center;padding:25px 38px;margin-top:10px;box-shadow:0 2px 8px #233b5204}.welcome-card h2{font-size:20px
;margin:8px 0}.welcome-card p{font-size:11px;line-height:1.8;color:#8290a0;max-width:590px}.welcome-art
{height:190px;width:190px;border-radius:50%;background:radial-gradient(circle at 50% 50%,#eeefff 0,#
f4f3ff 43%,#fbfbff 72%);display:grid;place-items:center;position:relative;color:#595bd0;margin-left:6px
```

```
    }.welcome-art>span{font-size:65px;line-height:1}.orbit{position:absolute;border:1px solid #bbb9f0;
    border-radius:50%;inset:23px 7px;transform:rotate(-30deg)}.orbit-two{inset:7px 24px;transform:rotate(35
    deg);border-color:#d3d1f6}.spark{font-style:normal;position:absolute;color:#e4a159}.spark-one{left:30px
    ;top:38px}.spark-two{right:25px;bottom:36px;font-size:12px}.welcome-steps{display:flex;gap:24px;margin-
    top:23px}.welcome-steps span{font-size:9px;color:#718093}.welcome-steps b{color:#595bd0;font-size:9px;
    margin-right:5px}.footer-note{display:flex;align-items:flex-start;gap:9px;padding:4px 3px;color:#a4aeb8
    }.footer-note span{font-size:14px;color:#716fd0}.footer-note p{font-size:9px;line-height:1.6;margin:0;
    max-width:880px}.error-banner{display:flex;align-items:center;gap:9px;background:#fff4f0;border:1px
    solid #f2d5cb;color:#aa5844;padding:11px 13px;border-radius:7px;margin:-10px 0 20px;font-size:11px}.
    error-banner>span{background:#b96851;color:white;width:17px;height:17px;border-radius:50%;display:grid;
    place-items:center;font-weight:700;font-size:10px}.error-banner button{margin-left:auto;border:0;
    background:none;color:#aa5844;font-size:18px}.spinner{width:12px;height:12px;border:2px solid #ffffff77
    ;border-top-color:#fff;border-radius:50%;display:inline-block;vertical-align:-2px;animation:spin .7s
    linear infinite}.spinner.dark{border-color:#dbe6e6;border-top-color:#128f84}@keyframes spin{to{
    transform:rotate(360deg)}}

@media(max-width:1250px){.sidebar{width:215px}.main-content{margin-left:215px}.page-wrap{padding:30px 28px}.
    topbar{padding:0 28px}.flagged-grid{grid-template-columns:repeat(3,minmax(0,1fr))}.source-card{grid-
    template-columns:40px 1fr}.source-controls{grid-column:2}.source-note{grid-column:2}.metrics-grid{grid-
    template-columns:repeat(2,1fr)}}
@media(max-width:760px){.sidebar{width:57px;padding:20px 8px}.brand{padding:0 4px 45px}.brand>span:last-
    child,.workspace-label,.side-caption,.nav-item:not(.active),.nav-item.active{font-size:0}.brand-mark{
    flex-shrink:0}.nav-item.active{width:40px;height:40px;padding:10px;font-size:0;justify-content:center}.
    nav-icon{font-size:17px}.side-rule{margin:19px 3px}.model-nav{padding:9px 16px;font-size:0}.sidebar-
    bottom{padding:15px 4px}.sidebar-bottom>div:last-child{display:none}.main-content{margin-left:57px}.
    topbar{height:54px;padding:0 17px}.page-wrap{padding:25px 16px}.page-heading{align-items:flex-start}.
    page-heading h1{font-size:23px}.page-heading p{font-size:10px;max-width:390px;line-height:1.6}.date-
    chip{display:none}.source-card{grid-template-columns:36px 1fr;padding:14px;gap:10px}.source-controls{
    grid-column:1/-1;flex-direction:column}.source-controls input,.primary-button{height:38px}.source-note{
    grid-column:1/-1}.metrics-grid{gap:8px}.metric-card{padding:12px;min-height:105px}.metric-card strong{
    font-size:21px}.metric-top{font-size:7px}.metric-icon{height:23px;width:23px}.flagged-grid{grid-
    template-columns:repeat(2,minmax(0,1fr));gap:8px}.flag-card{padding:12px 10px}.section-heading h2{font-
    size:15px}.section-heading p{max-width:280px;line-height:1.5}.directory-heading{align-items:flex-start;
    flex-direction:column;gap:12px}.directory-controls{width:100%}.search-box{flex:1}.search-box input{
    width:100%;min-width:0}.welcome-card{grid-template-columns:1fr;padding:24px;text-align:center}.welcome-
    art{height:145px;width:145px;margin:0 auto 20px}.welcome-steps{justify-content:center;gap:10px;flex-
    wrap:wrap}.detail-columns{grid-template-columns:1fr;gap:23px}.score-row{grid-template-columns:115px 1fr
     33px}.top-right{gap:8px}.local-badge{font-size:9px}.footer-note p{font-size:8px}}

.folder-picker{position:relative;display:flex;align-items:center;min-width:0;flex:1;height:38px;border:1px
    solid #dfe6eb;border-radius:7px;background:#fcfdfd;color:#657487;cursor:pointer;overflow:hidden}.folder
    -picker:hover{border-color:#7bc4ba;background:#f8fcfb}.folder-picker>span{overflow:hidden;text-overflow
    :ellipsis;white-space:nowrap;padding:0 12px;font-size:11px}.folder-picker input{position:absolute;inset
    :0;width:100%;height:100%;opacity:0;cursor:pointer}

.model-guide{background:#fff;border:1px solid #e4e7f2;border-radius:11px;margin:-7px 0 29px;box-shadow:0 2
    px 8px #24244708;overflow:hidden}.model-guide>summary{list-style:none;cursor:pointer;display:flex;align
    -items:center;justify-content:space-between;gap:18px;padding:16px 19px}.model-guide>summary::-webkit-
    details-marker{display:none}.model-guide>summary>span:first-child b,.model-guide>summary>span:first-
    child small{display:block}.model-guide>summary>span:first-child b{font-size:12px;color:#29304b}.model-
    guide>summary>span:first-child small{font-size:10px;color:#8993a7;margin-top:4px}.guide-action{font-
    size:10px;color:#5a5bc9;white-space:nowrap}.guide-action i{font-style:normal;font-size:15px;margin-left
    :8px}.model-guide[open]>summary{border-bottom:1px solid #ececf5}.model-guide[open] .guide-action i{
    display:inline-block;transform:rotate(45deg)}.guide-content{padding:17px 19px 19px}.flow-strip{display:
    grid;grid-template-columns:1fr auto 1fr auto 1fr auto 1fr;align-items:center;gap:12px;padding:14px;
    background:#f7f7fd;border:1px solid #ededf7;border-radius:9px}.flow-step{min-width:0}.flow-step small{
    display:block;font-size:8px;font-weight:700;letter-spacing:.8px;color:#7372c9;margin-bottom:7px}.flow-
    step b{display:block;font:700 11px Manrope,sans-serif;color:#343a56;margin-bottom:4px}.flow-step span{
    display:block;font-size:9px;line-height:1.5;color:#8791a3}.flow-arrow{font-style:normal;color:#a3a1d7;
```

```
    font-size:16px}.guide-section-title{display:flex;justify-content:space-between;align-items:baseline;gap
    :10px;margin:19px 1px 10px}.guide-section-title b{font-size:11px;color:#303750}.guide-section-title
    span{font-size:9px;color:#929bad}.model-explain-grid{display:grid;grid-template-columns:repeat(4,minmax
    (0,1fr));gap:9px}.model-explain{display:flex;align-items:flex-start;gap:9px;border:1px solid #ebedf4;
    border-radius:8px;padding:12px 10px;min-width:0}.model-explain>span{height:23px;width:23px;flex:0 0 23
    px;border-radius:7px;display:grid;place-items:center;font:700 9px Manrope,sans-serif}.model-explain
    small{font-size:7px;letter-spacing:.6px;font-weight:700;color:#8a94a6}.model-explain h3{font:700 11px
    Manrope,sans-serif;color:#303750;margin:5px 0}.model-explain p{font-size:9px;line-height:1.55;color
    :#788498;margin:0}.model-explain.indigo>span{background:#eeeeff;color:#5b5dd2}.model-explain.sky>span{
    background:#eaf5fc;color:#4581ae}.model-explain.violet>span{background:#f2edfc;color:#8667bd}.model-
    explain.amber>span{background:#fff3e8;color:#c47a38}.guide-rule{display:grid;grid-template-columns:175
    px 1fr;gap:12px;align-items:start;border-left:3px solid #696bd8;background:#f8f8fd;padding:11px 13px;
    margin-top:12px;border-radius:0 7px 7px 0}.guide-rule span{font-size:9px;font-weight:700;color:#5557b8
    }.guide-rule p{font-size:9px;line-height:1.6;color:#737f92;margin:0}

@media(max-width:1250px){.model-explain-grid{grid-template-columns:repeat(2,minmax(0,1fr))}.flow-strip{gap
    :7px}.flow-step span{font-size:8px}}
@media(max-width:760px){.model-guide>summary{padding:14px;align-items:flex-start}.guide-action{font-size:0}.
    guide-action i{font-size:17px}.guide-content{padding:13px}.flow-strip{grid-template-columns:1fr;gap:8px
    }.flow-arrow{transform:rotate(90deg);justify-self:center;font-size:13px}.flow-step{padding:4px 2px}.
    model-explain-grid{grid-template-columns:1fr}.guide-section-title{align-items:flex-start;flex-direction
    :column;gap:4px}.guide-rule{grid-template-columns:1fr;gap:5px}}
```

Listing 10.6: Dashboard base styles

```
:root {
  color: #202a35;
  background: #f5f3ee;
  --ink: #172330;
  --muted: #78828a;
  --line: #e8e3d9;
  --teal: #b3925a;
  --teal-dark: #967342;
  --surface: #fffefa;
}

body,
.app-shell {
  background: #f5f3ee;
}

.sidebar {
  background: #111c27;
  color: #e8e3d8;
}

.brand-mark {
  background: #b3925a;
  color: #172330;
}

.brand-light {
  color: #c9b78f;
}

.workspace-label,
.side-caption {
  color: #9a9c99;
}

.nav-item {
  color: #c2c7c9;
}

.nav-item.active {
  background: #26313a;
  box-shadow: inset 3px 0 #c3a36c;
}

.side-rule,
.sidebar-bottom {
  border-color: #2d3942;
}

.privacy-icon {
  background: #26313a;
  color: #cfb783;
}

.sidebar-bottom b {
  color: #f0ede5;
}

```

```
.sidebar-bottom small {
  color: #a5acae;
}

.main-content {
  background: #f5f3ee;
}

.topbar {
  background: #fffefa;
  border-color: #e8e3d9;
}

.local-badge {
  background: #f0eee7;
  color: #59645f;
}

.local-badge i {
  background: #72927f;
}

.avatar {
  background: #eee8d9;
  color: #806b43;
}

.eyebrow {
  color: #9a8154;
}

.page-heading h1,
.section-heading h2,
.directory-heading h2,
.detail-heading h2,
.welcome-card h2 {
  color: #172330;
}

.date-chip,
.source-card,
.metric-card,
.flag-card,
.directory-card,
.detail-card,
.model-guide,
.welcome-card {
  background: #fffefa;
  border-color: #e8e3d9;
}

.date-chip {
  color: #666e70;
}

.source-card,
.metric-card,
.directory-card,
.detail-card,
```

```
.welcome-card {
  box-shadow: 0 8px 24px rgb(30 37 41 / 4%);
}

.source-icon,
.metric-icon.teal {
  background: #f0eadc;
  color: #967342;
}

.source-copy b,
.metric-card strong,
.employee-name,
.section-heading h2,
.directory-heading h2 {
  color: #202a35;
}

.source-controls input {
  background: #fffefa;
  border-color: #ded9cf;
}

.source-controls input:focus,
.search-box:focus-within {
  border-color: #b3925a;
  box-shadow: 0 0 0 3px rgb(179 146 90 / 15%);
}

.primary-button {
  background: #a98954;
  color: #fffefa;
  box-shadow: 0 3px 8px rgb(117 91 48 / 18%);
}

.primary-button:hover {
  background: #8f7141;
}

.flag-card:hover,
.flag-card.chosen {
  border-color: #c5aa78;
  box-shadow: 0 8px 22px rgb(74 62 39 / 9%);
}

.queue-count {
  background: #f6efe2;
  color: #896a39;
  border-color: #eadcc2;
}

.model-guide > summary {
  color: #25313b;
}

.flow-step,
.model-explain {
  border-color: #e9e4da;
}
```

```
177
178 .guide-rule {
179   background: #f7f4ed;
180   border-color: #e9e2d5;
181 }
182
183 .guide-rule > span {
184   color: #8f7141;
185 }
186
187 .guide-action,
188 .welcome-steps b,
189 .footer-note > span {
190   color: #967342;
191 }
192
193 .welcome-art {
194   background: #f1ebdf;
195   color: #a98954;
196   border-color: #c8b489;
197 }
198
199 .welcome-art > span,
200 .orbit {
201   color: #967342;
202   border-color: #c6b58f;
203 }
204
205 .welcome-steps b {
206   background: #f0eadc;
207 }
208
209 .footer-note > span {
210   background: #f0eadc;
211 }
212
213 .empty-state,
214 .table-empty {
215   background: #fffefa;
216   border-color: #e8e3d9;
217 }
218
219 ::selection {
220   color: #172330;
221   background: #e9dec7;
222 }
223
224 :root {
225   background: #f3f6fa;
226   --line: #e2e9f0;
227   --teal: #4779ad;
228   --teal-dark: #315f91;
229   --surface: #fff;
230 }
231
232 body,
233 .app-shell,
234 .main-content {
235   background: #f3f6fa;
```

```
}

.sidebar {
  background: #10243b;
}

.brand-mark {
  background: #4779ad;
  color: #fff;
}

.brand-light {
  color: #aec5dc;
}

.nav-item.active {
  background: #1d3550;
  box-shadow: inset 3px 0 #79a6d2;
}

.privacy-icon {
  background: #1d3550;
  color: #9fc0e0;
}

.topbar,
.date-chip,
.source-card,
.metric-card,
.flag-card,
.directory-card,
.detail-card,
.model-guide,
.welcome-card,
.empty-state,
.table-empty {
  background: #fff;
  border-color: #e2e9f0;
}

.local-badge {
  background: #edf2f7;
}

.avatar {
  background: #e4edf6;
  color: #315f91;
}

.eyebrow {
  color: #547ba3;
}

.source-icon,
.metric-icon.teal,
.welcome-steps b,
.footer-note > span {
  background: #e9f0f7;
  color: #396a9e;
```

```
}

.source-controls input {
  background: #fff;
  border-color: #d8e1eb;
}

.source-controls input:focus,
.search-box:focus-within {
  border-color: #6d99c3;
  box-shadow: 0 0 0 3px rgb(71 121 173 / 14%);
}

.primary-button {
  background: #396a9e;
  box-shadow: 0 3px 8px rgb(49 95 145 / 20%);
}

.primary-button:hover {
  background: #2d5785;
}

.queue-count {
  background: #edf3f9;
  color: #315f91;
  border-color: #d7e3ef;
}

.flag-card:hover,
.flag-card.chosen {
  border-color: #8eaccb;
  box-shadow: 0 8px 22px rgb(37 73 110 / 9%);
}

.guide-rule {
  background: #f2f6fa;
  border-color: #dfe8f1;
}

.agreement-note {
  display: flex;
  align-items: flex-start;
  gap: 11px;
  margin: -3px 0 14px;
  padding: 12px 14px;
  border: 1px solid #dfe8f1;
  border-radius: 9px;
  background: #f7fafd;
}

.agreement-note-icon {
  display: grid;
  flex: 0 0 25px;
  height: 25px;
  place-items: center;
  border-radius: 8px;
  color: #315f91;
  background: #e7eff7;
  font-size: 15px;
```

```
}

.agreement-note b { display: block; margin: 1px 0 4px; color: #40566d; font-size: 10px; }
.agreement-note p { margin: 0; max-width: 920px; color: #76879a; font-size: 9px; line-height: 1.65; }

.guide-rule > span,
.guide-action,
.welcome-steps b,
.footer-note > span {
  color: #315f91;
}

.welcome-art {
  background: #eaf1f8;
  color: #4779ad;
  border-color: #a9c1d9;
}

.welcome-art > span,
.orbit {
  color: #315f91;
  border-color: #a9c1d9;
}

::selection {
  color: #10243b;
  background: #d8e6f3;
}

.analysis-card {
  display: grid;
  grid-template-columns: 180px minmax(0, 1fr);
  align-items: center;
  gap: 28px;
  margin: -8px 0 24px;
  padding: 22px 28px;
  overflow: hidden;
  border: 1px solid #dce5ef;
  border-radius: 14px;
  background: radial-gradient(ellipse at 8% 50%, #edf4fc 0, #fff 56%);
  box-shadow: 0 8px 28px rgb(37 73 110 / 7%);
}

.analysis-visual {
  position: relative;
  width: 150px;
  height: 150px;
  margin: auto;
  display: grid;
  place-items: center;
}

.analysis-orbit {
  position: absolute;
  inset: 17px 0;
  border: 1px solid #b9cee2;
  border-radius: 50%;
  animation: analysis-rotate 7s linear infinite;
}
```

```
413
414 .analysis-orbit.orbit-b {
415   inset: 0 17px;
416   border-color: #d4e1ee;
417   animation-direction: reverse;
418   animation-duration: 9s;
419 }
420
421 .analysis-core {
422   display: grid;
423   width: 70px;
424   height: 70px;
425   place-items: center;
426   border: 1px solid #d2e0ed;
427   border-radius: 22px;
428   color: #315f91;
429   background: linear-gradient(145deg, #fff, #eaf2f9);
430   box-shadow: 0 8px 24px rgb(49 95 145 / 15%);
431   animation: analysis-float 2.4s ease-in-out infinite;
432 }
433
434 .analysis-core span { font-size: 37px; line-height: 1; }
435
436 .analysis-node {
437   position: absolute;
438   z-index: 1;
439   width: 10px;
440   height: 10px;
441   border: 2px solid #fff;
442   border-radius: 50%;
443   background: #4779ad;
444   box-shadow: 0 0 0 4px #4779ad20;
445   animation: analysis-twinkle 1.8s ease-in-out infinite;
446 }
447
448 .node-a { top: 15px; left: 48px; }
449 .node-b { top: 55px; right: 5px; animation-delay: .35s; }
450 .node-c { bottom: 17px; left: 50px; animation-delay: .7s; }
451 .node-d { top: 54px; left: 5px; animation-delay: 1.05s; }
452 .analysis-copy .eyebrow { color: #4779ad; }
453 .analysis-copy h2 { margin: 6px 0; color: #243346; font: 700 18px Manrope, sans-serif; }
454 .analysis-copy p { margin: 0 0 17px; color: #7d8b9b; font-size: 11px; }
455 .analysis-progress-heading { display: flex; justify-content: space-between; margin-bottom: 7px; color:
      #66778a; font-size: 10px; }
456 .analysis-progress-heading b { color: #315f91; font-variant-numeric: tabular-nums; }
457 .analysis-time-remaining { display: flex; align-items: center; gap: 7px; margin-top: 9px; color: #55708b;
      font-size: 10px; font-variant-numeric: tabular-nums; }
458 .analysis-time-remaining small { margin-left: auto; color: #a1adba; font-size: 9px; }
459 .time-pulse { width: 6px; height: 6px; border-radius: 50%; background: #6b94bb; box-shadow: 0 0 0 3px #6
      b94bb20; animation: analysis-twinkle 1.8s ease-in-out infinite; }
460 .analysis-progress-track { height: 8px; overflow: hidden; border-radius: 9px; background: #eaf0f6; }
461 .analysis-progress-track i { position: relative; display: block; height: 100%; min-width: 3px; border-
      radius: inherit; background: linear-gradient(90deg, #6c91b8, #315f91); transition: width .45s ease; }
462 .analysis-progress-track i::after { position: absolute; inset: 0; content: ''; background: linear-gradient
      (100deg, transparent 20%, #ffffff88 50%, transparent 80%); animation: analysis-sheen 1.5s ease-in-out
      infinite; }
463 .analysis-stage-list { display: flex; justify-content: space-between; margin-top: 9px; color: #a1adba; font-
      size: 9px; }
464 .analysis-stage-list span { display: flex; align-items: center; gap: 5px; }
```

```
.analysis-stage-list b { font-weight: 500; }
.analysis-stage-list .current { color: #315f91; }
.analysis-stage-list .complete { color: #58917f; }

@keyframes analysis-rotate { to { transform: rotate(360deg); } }
@keyframes analysis-float { 50% { transform: translateY(-4px); box-shadow: 0 12px 28px rgb(49 95 145 / 22%);
      } }
@keyframes analysis-twinkle { 50% { transform: scale(1.35); opacity: .65; } }
@keyframes analysis-sheen { from { transform: translateX(-110%); } to { transform: translateX(110%); } }

@media (max-width: 600px) {
  .analysis-card { grid-template-columns: 1fr; gap: 8px; padding: 17px; }
  .analysis-visual { width: 105px; height: 105px; }
  .analysis-core { width: 55px; height: 55px; border-radius: 17px; }
  .analysis-core span { font-size: 29px; }
}

@media (prefers-reduced-motion: reduce) {
  .analysis-orbit, .analysis-core, .analysis-node, .analysis-progress-track i::after { animation: none; }
  .analysis-progress-track i { transition: none; }
}
```

Listing 10.7: Dashboard theme styles

11. References

Bibliography

[1] Software Engineering Institute, Carnegie Mellon University, *CERT Insider Threat Test Dataset, Version 4.2*. Available: https://www.sei.cmu.edu/library/insider-threat-test-dataset/. Accessed: 25 October 2026.

[2] Joint Task Force, *Security and Privacy Controls for Information Systems and Organizations*, NIST Special Publication 800-53, Revision 5, National Institute of Standards and Technology, September 2020. doi: https://doi.org/10.6028/NIST.SP.800-53r5.

[3] P. Cichonski, T. Millar, T. Grance, and K. Scarfone, *Computer Security Incident Handling Guide*, NIST Special Publication 800-61, Revision 2, National Institute of Standards and Technology, August 2012. doi: https://doi.org/10.6028/NIST.SP.800-61r2.

[4] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011. Available: https://jmlr.org/papers/v12/pedregosa11a.html.

[5] The pandas development team, *pandas documentation*. Available: https://pandas.pydata.org/docs/. Accessed: 25 October 2026.

[6] S. Ramirez, *FastAPI documentation*. Available: https://fastapi.tiangolo.com/. Accessed: 25 October 2026.

[7] Meta Open Source, *React documentation*. Available: https://react.dev/. Accessed: 25 October 2026.

[8] Python Software Foundation, *Python 3 documentation*. Available: https://docs.python.org/3/. Accessed: 25 October 2026.

[9] National Institute of Standards and Technology, *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*, NIST AI 100-1, January 2023. doi: https://doi.org/10.6028/NIST.AI.100-1.